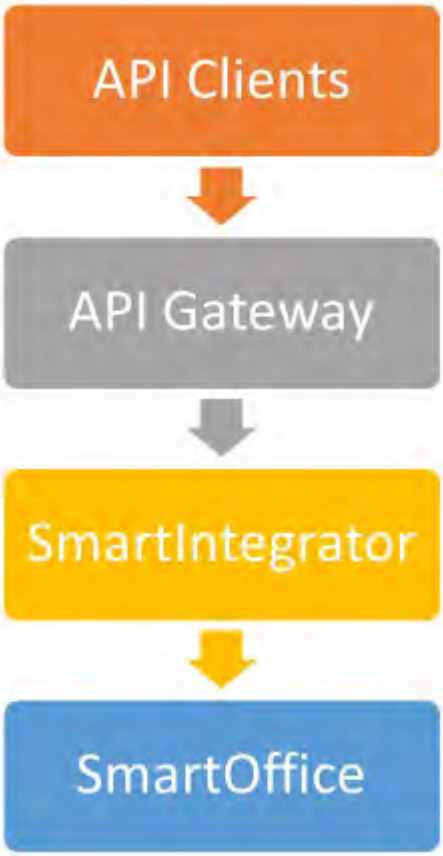
Introduction

The SmartIntegrator components provide a standard method for exchanging data between SmartOffice® and other applications.

Data is exchanged with client applications in a request/response fashion using standard XML.

SmartIntegrator is accessed through an API gateway. The gateway authenticates requests using an API key and secret key and then passes the request to SmartIntegrator, which then returns a response.



SmartIntegrator Architecture

Developers are provided with an API key and a secret key as part of the developer onboarding process; both keys expire after a specific period and need to be regenerated.

XML Data (Object/Property)

In this Topic Hide

- Overview
- Object
- Property
  - Simple Value
  - Child Object
  - Child Object Array
  - ID
  - Link Type
- Value Format

Overview

SmartIntegrator's XML data is organized as **Object** and **Property** elements. Object elements represent data entities that contain a set of property elements. Property elements usually represent sample values but can also contain an object or an object array.

Example:

```
<Contact id="Contact.1.1">
  <LastName>Ackerman</LastName>
  <FirstName>Joe</FirstName>
  <PreferredPhone>
     <AreaCode>626</AreaCode>
     <Number>5553505</Number>
  </PreferredPhone>
</Contact>
```

Object

Each object has a globally unique ID. The ID is stored in an object's `id` attribute. Objects may also have child objects. Child objects are embedded within the parent object's property elements.

Property

Property elements are child elements of an object element. There are five property types recognized by SmartIntegrator:

Simple Value

A property element may contain only a simple text value. For example:

```
<LastName>Ackerman</LastName>
```

Child Object

A property element can represents another object and contain its own properties. For example:

```
<Contact id="Contact.1.1">
  <PreferredPhone>
     <AreaCode>626</AreaCode>
     <Number>5553505</Number>
  </PreferredPhone>
</Contact>
```

Child Object Array

This kind of property element is an array of an object. For example:

```
<Contact id="Contact.1.1">
  <Phones>
     <Phone>
        <AreaCode>626</AreaCode>
        <Number>5553505</Number>
     </Phone>
  </Phones>
</Contact>
```

ID

An ID property uniquely identifies an object and can only reference that one type of object. For example:

```
<PreferredPhoneId>Phone.1.1</PreferredPhoneId>
```

Link Type

The Link Type property can reference different types of objects. For example:

```
<SmartPad id="SmartPad.65854.111">
  <link>Activity.65854.125</link>
</SmartPad>
<SmartPad id="SmartPad.65854.112">
  <link>Policy.65854.125</link>
</SmartPad>
```

Value Format

SmartIntegrator uses standard XML to format values for request and response. Acceptable data types include:

Type	Format	Examples
Boolean	1 (true), 0 (false)	
String		
Long, Integer, Short, Byte	Number format ########	
Decimal, Double, Float	######.####	
Datetime	CCYY-MM-DD hh:mm:ssZ	2002-09-02 23:20:30-0700
Date	CCYY-MM-DD	2002-09-02
Time, Duration	hh:mm:ssZ	23:20:30-0700
Binary	Base 64 encoding	

Request/Response

In this Topic Hide

- Overview
- Request XML
  - Header Variable Values
  - Authentication
  - Transaction Element
  - User Attributes
- Response XML

Overview

Client applications send a request XML to SmartIntegrator and receive a response in XML format. The request XML includes actions, data and authentication information. The response XML from SmartIntegrator includes status and result data.

XML requests and responses contain two components: a header and content. The header contains authentication information.

Once you understand how requests are formatted, refer to the operations you can carry out using SmartIntegrator.

Request XML

The following example request uses the `get` operation to retrieve a contact's first and last names:

```
<request version="1.0">
  <header>
     <office/>
     <user/>
     <password/>
  </header>
  <get>
     <Contact id="Contact.1.1">
        <LastName/>
        <FirstName/>
```

```
    </Contact>
  </get>
</request>
```

Important: The `header` element of every request must include the `office`, `user` and `password` as empty elements, as shown in the example above. However, to simplify examples elsewhere in this documentation, the abbreviation `<header>...</header>` is used.

Header Variable Values

All requests send during development of your integration must contain the headers variables described in the table below. These values are pre-filled in the Postman collection you received during the onboarding process.

Header Variable	Description	Required
sitename	SmartOffice site name	Yes
username	SmartOffice user name in the format `<SiteName>_<OfficeName>_<UserName>` (the client needs to know for which office and user the request needs to execute)	Yes
api-key	Key (learn more)	Yes
api-secret	Key (learn more)	Yes

The API gateway processes the request XML as a stream. This requires that the header be the first child of the root element. Otherwise, the engine will stop processing and an error response XML will be returned.

Authentication

Authentication information is stored in the Headers section in the Postman collection you received during the onboarding process.

Transaction Element

Each coherent operation can be specified within a single `transaction` element. A request can contain multiple transactions. The following example shows the basic structure of a request containing a `transaction` element and two operations, `insert` and `update`.

```
<request>
<header>...</header>
<transaction>
  <insert>...</insert>
  <update>...</update>
</transaction>
</request>
```

Using the `transaction` element in a request is optional. If no transaction is required, the `transaction` element can be omitted, and the query can be sent directly with the query element at the same level as the header.

Use of the `transaction` element is recommended when you are performing operations (like `insert` or `update`) that can cause data issues if they are interrupted by connection problems or other issues. SmartIntegrator processes all operations within the `transaction` element before making any changes to the database.

User Attributes

Attributes of key elements (`request`, `header`, `transaction`, `insert`) included in a request are written back in the response XML. This feature enables a client application to place marks, or "signatures," in the request XML and trace them back through the response XML. In this way, the caller can locate the targets in a response XML stream. Note that user attributes are ignored if added to object and property elements.

In the following example request, `traceid` and `mark` are user attributes.

```
<request version="1.0" traceid="123341254">
  <header>...</header>
  <get mark="abc">
    <Contact id="Contact.65854.3">
      <FullName/>
    </Contact>
  </get>
</request>
```

SmartIntegrator copies them to the response XML:

```
<response version="1.0" traceid="123341254">
  <header>
  </header>
  <get mark="abc">
    <Contact id="Contact.65854.3">
      <FullName>Ackerman,Joseph</FullName>
    </Contact>
  </get>
  <!-- Cost 47 mill seconds. -->
</response>
```

Response XML

The content of an XML response shows the results of actions. The response XML ordinarily has a similar structure as a request XML.

Example:

```
<response version="1.0">
  <header>
  </header>
  <get>
    <Contact id="Contact.1.1">
      <LastName>Ackerman</LastName>
      <FirstName>Joe</FirstName>
    </Contact>
  </get>
</response>
```

Operations

Seven different operations can be used in the request XML: `insert`, `update`, `delete`, `get`, `search`, `method` and `sync`.

Data elements are children of operation elements. Operations can manipulate an object or call a method. Object manipulation operations call SmartOffice data beans internally.

The `method` operation exposes methods from SmartOffice's internal business functions. Client applications can use the `method` operation to perform actions based on predefined business rules.

Actions on object elements under operation elements are performed with the specified operation. A single operation can have multiple data objects.

Click below for more information about each operation:

- insert
- update
- delete
- get
- search
- method
- sync

Insert Operation

The `insert` operation creates objects and their child objects. The ID of the new object is returned in the response XML.

Tip: Enclosing `insert` requests in the `transaction` element is recommended to prevent data issues caused by connection interruptions or other problems.

The following example `insert` request creates a contact object and a child object (preferred phone number):

```
<request version="1.0">
  <header>...</header>
  <insert>
    <Contact>
      <LastName>Ackerman</LastName>
      <FirstName>Joe</FirstName>
      <PreferredPhone>
        <AreaCode>626</AreaCode>
        <Number>5853505</Number>
      </PreferredPhone>
    </Contact>
  </insert>
</request>
```

The response XML returns the status of the requested operation and the IDs of the new object and its child object:

```
<response version="1.0">
  <header>...</header>
  <insert>
    <Contact id="Contact.123.456" _status="inserted">
      <PreferredPhone id="Phone.123.789" _status="inserted"/>
    </Contact>
  </insert>
</response>
```

Update Operation

In this Topic Hide

- Overview
- Basic Usage
- Updating Objects in Arrays

Overview

The `update` operation modifies existing objects and their child objects. The request must specify a valid ID for the main (topmost) parent object.

Note: The `update` operation can update objects by ID only and does not support update by condition.

For a child object, the ID is not required. The child object will be inserted if it does not exist.

Tip: Enclosing `update` requests in the `transaction` element is recommended to prevent data issues caused by connection interruptions or other problems.

Basic Usage

The following example request updates the last name and the area code of the preferred phone number for a contact object:

```
<request version="1.0">
  <header>...</header>
  <update>
    <Contact id="Contact.123.456">
      <LastName>Ackerman</LastName>
      <PreferredPhone>
        <AreaCode>818</AreaCode>
      </PreferredPhone>
    </Contact>
  </update>
</request>
```

If an object already exists (in this example, the PreferredPhone object), the response looks like this:

```
<response version="1.0">
  <header>...</header>
```

```
  <update>
    <Contact id="Contact.123.456" _status="updated">
      <PreferredPhone id="Phone.123.789" _status="updated"/>
    </Contact>
  </update>
</response>
```

If the PreferredPhone object does not exist, the response shows that a phone object was inserted:

```
<response version="1.0">
  <header>...</header>
  <update>
    <Contact id="Contact.123.456" _status="updated">
      <PreferredPhone id="Phone.123.789" _status="inserted" />
    </Contact>
  </update>
</response>
```

Updating Objects in Arrays

To update an object in a child object array, you must include an ID for that object in the request. Otherwise, you will insert a new object in the array.

Request with ID: The following example request uses an ID to update the area code and phone number of an existing phone number object:

```
<request version="1.0">
  <header>...</header>
  <update>
    <Contact id="Contact.123.456">
      <Phones>
        <Phone id="Phone.123.789">
          <AreaCode>626</AreaCode>
          <Number>5553550</Number>
        </Phone>
      </Phones>
    </Contact>
  </update>
</request>
```

The response indicates that the specified object in the child object array was updated:

```
<response version="1.0">
  <header>...</header>
  <update>
    <Contact id="Contact.123.456">
      <Phones>
        <Phone id="Phone.123.789" _status="updated"/>
      </Phones>
```

```
    </Contact>
  </update>
</response>
```

Request without ID: The following example request does not specify an ID for the phone number:

```
<request version="1.0">
  <header>...</header>
  <update>
    <Contact id="Contact.123.456">
      <Phones>
        <Phone>
          <AreaCode>626</AreaCode>
          <Number>5553550</Number>
        </Phone>
      </Phones>
    </Contact>
  </update>
</request>
```

In this instance, the response indicates that a new phone number object was inserted into the database:

```
<response version="1.0">
  <header>...</header>
  <update>
    <Contact id="Contact.123.456">
      <Phones>
        <Phone id="Phone.123.790" _status="inserted" />
      </Phones>
    </Contact>
  </update>
</response>
```

Delete Operation

The `delete` operation removes objects. This operation requires an object ID. Only the main object and its ID need to be specified. Child objects are ignored.

This example deletes a contact object:

```
<request version="1.0">
  <header>...</header>
  <delete>
    <Contact id="Contact.123.456"/>
  </delete>
```

```
</request>
```

The response confirms the deletion was successful:

```
<response version="1.0">
  <header>...</header>
  <delete>
    <Contact id="Contact.123.456" _status="deleted" />
  </delete>
</response>
```

Get Operation

In this Topic Hide

- Overview
- Get Lookup

Overview

The get operation retrieves an object and its child objects' information by ID. The main object's ID is required. The child object's ID is not required.

A get request must also specify the properties to be returned in the response. If no properties are specified, the response will contain the default properties. The object ID is always returned in the response.

The following example request retrieves a contact's last name, first name and preferred phone number (area code and number):

```
<request version="1.0">
  <header>...</header>
  <get>
    <Contact id="Contact.123.456">
      <LastName/>
      <FirstName/>
      <PreferredPhone>
        <AreaCode/>
        <Number/>
      </PreferredPhone>
    </Contact>
  </get>
</request>
```

The response containing the requested data looks like this:

```
<response version="1.0">
  <header>...</header>
  <get>
    <Contact id="Contact.123.456">
      <LastName>Ackerman</LastName>
      <FirstName>Joe</FirstName>
      <PreferredPhone id="Phone.123.789">
        <AreaCode>626</AreaCode>
        <Number>5553505</Number>
      </PreferredPhone>
    </Contact>
  </get>
</response>
```

Get Lookup

SmartOffice drop-down lists are represented as lookups in SmartIntegrator. The get lookup operation retrieves the index and description of each choice that exists for a particular lookup, based on the lookup ID.

This example requests the indexes and descriptions for two lookups:

```
<request version="1.0">
  <header>...</header>
  <get>
    <lookup id="3"/>
    <lookup id="4"/>
  </get>
</request>
```

The response returns the index and description (if available) for each lookups:

```
<response version="1.0">
  <header>...</header>
  <get>
    <lookup name="YesNoUnknown" id="3">
      <value index="0"/>
      <value index="1">Yes</value>
      <value index="2">No</value>
    </lookup>
    <lookup name="Record Type" id="4">
      <value index="1">Individual</value>
      <value index="2">Business</value>
      <value index="10">Archived</value>
    </lookup>
  </get>
</response>
```

Search Operation

In this Topic Hide

- Overview
- Search Attributes
- Object Element
- Condition Element
- Sort Element
- Pagination

Overview

The search operation makes use of numerous attributes and conditions to locate objects in the SmartOffice database.

The example request below shows a typical search operation. It finds all contacts that have a last name beginning with "Ack"; returns the ID, last name and first name of each contact; sorts the results by first name; returns 20 "pages" of results at a time; and stops returning results after 20 "pages."

For explanations of the various elements and attributes, continue to the next section.

```
<request version="1.0">
  <header>...</header>
  <search maxpages="200" pagesize="20">
    <object>
      <Contact>
        <LastName/>
        <FirstName/>
      </Contact>
    </object>
    <condition>
      <expr prop="LastName" op="starts">
        <v>Ack</v>
      </expr>
    </condition>
    <sort>
```

```
      <item prop="FirstName" order="asc"/>
    </sort>
  </search>
</request>
```

The response includes the requested object information:

```
<response version="1.0">
  <header>...</header>
  <search pagesize="20" page="0" total="1" more="false">
    <Contact id="Contact.123.456">
      <LastName>Ackerman</LastName>
      <FirstName>Joe</FirstName>
    </Contact>
  </search>
</response>
```

Note the search element attributes in the response. These provide information about the current page of results, the total number or records returned in the search, and whether there are more pages.
See Pagination for more details.

Search Attributes

The following table describes the attributes that can be used in the search element.

Attribute	Description
searchid	Search id, read-only. Specifies the unique ID of the search. The searchid is necessary when the search results are on multiple pages (see Pagination for more details).
pagesize	Number of objects each page, read/write. The default page size is 2000.
maxpages	The maximum number of pages the search should return.
page	Current page, read/write. If the results are greater than the pagesize, this attribute lists the current page number to note where you are.

total	Number of records retrieved in the response, read-only.
more	Specifies whether there are more records to be retrieved from the search, read-only. See Pagination for more details.

Object Element

The object element of a search operation is a template object element that specifies the main object and properties that should be returned in the response. If no properties are specified in the object element, SmartIntegrator returns the default properties.

Condition Element

The condition element of a search operation, which is required, specifies the conditions of the query, which are defined using one or more expr elements.

For more information, see Conditions.

Sort Element

See Sort.

Pagination

When you need to retrieve a large amount of data in a search operation, SmartIntegrator provides the ability to produce paginated data .

Using the search ID generated after the first search, a client application can retrieve a selected element's searchid attribute from the response XML and specify that searchid and the desired page number in a subsequent request to retrieve another page of results.

Not specifying a page number or leaving it blank defaults to retrieving the next page. Clients will not know the total number of records until the last

page is retrieved, in which case the more attribute of the search element in the response will be false).

Use of pagination is highly recommended for better performance and efficient data retrieval.

The following example shows a search request with pagination.

Important: The keepsession element with a value of true is required in the header element for pagination to work. WIthout it, you will not receive a searchid value in the response and will not be able to retrieve subsequent pages.

```
<request version="1.0">
  <header>
    <office/>
    <user/>
    <password/>
    <keepsession>true</keepsession>
  </header>
  <search total="true" pagesize="5">
    <object>
      <Contact>
        <LastName/>
        <FirstName/>
      </Contact>
    </object>
    <condition>
      <expr prop='LastName' op='starts'>
        <v>A</v>
      </expr>
    </condition>
  </search>
</request>
```

The response returns five records (due to the pagesize attribute in the request) and provides the search ID for retrieving more pages:

```
<response version="1.0">
  <header/>
  <search total="6" searchid="MQ==" more="true" pagesize="5" office="MYSO"
page="0" user="Test">
    <Contact_type="obj" id="Contact.90807498.174">
      <LastName>Aetna Senior Supplement</LastName>
    </Contact>
    <Contact_type="obj" id="Contact.90807498.163320593">
      <LastName>American Funds</LastName>
    </Contact>
    <Contact_type="obj" id="Contact.90807498.163321105">
      <LastName>A</LastName>
    </Contact>
    <Contact_type="obj" id="Contact.90807498.163321733">
      <LastName>Advantage Elite Select 30</LastName>
    </Contact>
```

```
    <Contact_type="obj" id="Contact.90807498.163321884">
      <LastName>AdamsBridge</LastName>
    </Contact>
  </search>
</response>
```

The follow-up request includes the search ID.

```
<request version="1.0">
  <header>
    <office/>
    <user/>
    <password/>
    <keepsession>true</keepsession>
  </header>
  <search total="true" pagesize="5" searchid="MQ==">
    <object>
      <Contact>
        <LastName/>
        <FirstName/>
      </Contact>
    </object>
    <condition>
      <expr prop='LastName' op='starts'>
        <v>A</v>
      </expr>
    </condition>
  </search>
</request>
```

Because the request does not include the page attribute, SmartIntegrator returns the next page by default:

```
<response version="1.0">
  <header/>
  <search total="6" searchid="MQ==" more="false" pagesize="5" office="MYSO"
page="-1" user="Test">
    <Contact _type="obj" id="Contact.90807498.163321887">
      <LastName>Appollo Hospital</LastName>
    </Contact>
  </search>
</response>
```

Method Operation

In this Topic Hide

- Overview
- Documented Methods

Overview

The method operation enables client applications to make calls to SmartOffice's predefined business rules. SmartIntegrator parses the request XML to get all the parameters and then calls the corresponding method before finally wrapping the result in the response XML.

Each method has different parameters and return values. All available methods are listed in the API Dictionary; many of the most commonly used methods are documented here.

The following example method request sends a request to the GetContactDOB method, which returns all contacts whose birth dates fall within a specified date range and who are assigned to a particular SmartOffice user:

```
<request version="1.0">
  <header>...</header>
  <method>
    <GetContactDOB>
      <UserID>User.1.2</UserID>
      <FromDate>1950-01-01</FromDate>
      <ToDate>1955-12-31</ToDate>
    </GetContactDOB>
  </method>
</request>
```

In this instance, the response returns two contacts:

```
<response version="1.0">
  <header/>
  <method>
    <GetContactDOB>
      <Contact id="Contact.1.12" _type="obj">
        <FirstName>Paulette</FirstName>
        <LastName>Garner</LastName>
        <Person id="Person.1.12" _type="obj">
          <Dob>1952-07-03</Dob>
        </Person>
      </Contact>
      <Contact id="Contact.1.14" _type="obj">
        <FirstName>Genevieve</FirstName>
        <LastName>Bates</LastName>
        <Person id="Person.1.14" _type="obj">
          <Dob>1954-10-16</Dob>
        </Person>
      </Contact>
    </GetContactDOB>
  </method>
```

```
</response>
```

Documented Methods

This section links to documentation covering many commonly used methods.

This is not an exhaustive list of available methods. For a complete list, refer to the API Dictionary.

Method Name	Description
AssignContactstoAgent	Assigns a SmartView for Advisors user to multiple contact records under specific conditions.
CheckDuplicateContact	Checks the contact information submitted in the request against existing contacts in the SmartOffice database to determine whether a matching contact exists.
`CreateSVAdvisorUser`	Creates a SmartView for Advisors user account for a specified advisor contact.
DelContact	Removes a contact from a set.
`DelContactSubObjs`	Deletes all phone number, address and e-mail/web address entries from a contact record.
DeleteActivityContact	Removes a contact from a calendar activity.
`DisableUser`	Changes the status of a SmartOffice user account to Disabled.

DynamicReport	Retrieve the current data for a Dynamic Report.
DynamicReportFilter	Retrieves the default filter syntax of a Dynamic Report.
DynamicReportFilterDetail	Returns the detailed default filter syntax of a Dynamic Report.
GetActColors	Returns a list of the colors used to help identify activity types and subtypes in the calendar.
GetAdvisorCommission	Retrieves all payable commission records linked to a specified SmartView for Advisors user account.
GetConfiguration	Retrieves the content of an element in the SmartOffice server configuration file.
GetContactDOB	Returns contacts whose birth dates fall within a specified date range and who are assigned to a particular SmartOffice user.
GetContactsForEmail	Returns all contacts in SmartOffice who have a specified e-mail addresses.
GetCustomLabelDetail	Returns the label assigned to a particular custom field for a particular SmartOffice module and record type.
GetCustomLabels	Returns a list of all custom field labels for a particular SmartOffice module and record type.

GetDeletedContacts	Returns the object IDs of all contacts that exist in the Deleted Records area of SmartOffice.
GetDeletedPolicies	Returns the object IDs of all insurance policies that exist in the Deleted Records area of SmartOffice.
GetHouseholdMember	Retreives a list of all members of a contact's household.
GetInvtMaturityDate	Retrieves all investment positions that will mature within a specified date range for all contacts assigned to a particular SmartOffice user.
GetOfficeMultiFactorAuthentication	Indicates whether an office has two-step authentication enabled.
GetPolicyInvtData	Retrieves details of a specified contact's policy and investment holdings.
GetSiteName	Returns the name of the site where the SmartOffice server is hosted.
GetSystemTime	Returns the current date and time on the SmartOffice server.
GetUnassignedAccounts	Returns the object IDs of all investment accounts in an office that have a particular account number and are not assigned to a contact.
GetUserLicense	Returns the names and IDs of licenses enabled for a particular SmartOffice user.

GetUserMultiFactorAuthentication	Indicates whether a user account has two-step authentication enabled.
InsertKeyRelations	Adds a key relation to a contact record.
InsertRequirement	Generates requirements for a policy or pending case based on the requirement guidelines defined for the associated carrier and/or product.
ProcessCustomChoice	Inserts, modifies or deletes custom lookup choices.
ProcessSmartPadArchive	Archives a SmartPad entry or restores an archived SmartPad entry.

AssignContactstoAgent Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The AssignContactstoAgent method mass assigns a SmartView for Advisors user to multiple contact records under specific conditions. This method works when office security is enabled.

The AssignContactstoAgent method is invoked using the method operation.

The user invoking this method must have the Mass Assignments user right in SmartOffice.

A user assignment is created successfully when any of the following conditions exist:

- The SmartView for Advisors user is the contact's primary advisor.
- The SmartView for Advisors user is an advisor on any of the contact's policies or investment accounts.

If none of these conditions are met, or if the request parameters contain incorrect values, the XML response contains the message "No contacts for Assignment."

Parameters

Parameter Name	Type	Required?	Description
AgentID	String	Yes	This is the ID of the advisor record linked to the SmartView for Advisors user account. Note that this value is the primary key (e.g., 112), not the full object ID (e.g., Agent.1.112).
AgentOfficeID	String	Yes	This is the ID of the office in which the SmartView for Advisors user account resides. Note that this value is the primary key (e.g., 1), not the full object ID (e.g., Office.1).

Example

The following example request assigns a SmartView for Advisors user (whose agent ID is 112) to all contacts that meet the method's assignment conditions:

```
<request version='1.0'>
    <header>...</header>
    <method>
        <AssignContactstoAgent>
            <AgentID>112</AgentID>
            <AgentOfficeID>1</AgentOfficeID>
```

```
        </AssignContactstoAgent>
    </method>
</request>
```

The response includes the advisor's object ID and a confirmation message:

```
<response version="1.0">
    <header>
        <sessionClosed/>
    </header>
    <method>
        <AssignContactstoAgent>
            <Response id="Agent.1.112">Contact Assignment is Done</Response>
        </AssignContactstoAgent>
    </method>
    <_status>OK</_status>
    <_systime>2015-10-09T22:49:16</_systime>
    <!--Cost 90 mill seconds.-->
</response>
```

CheckDuplicateContact Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The CheckDuplicateContact method checks the contact information submitted in the request XML against existing contacts in the SmartOffice database. The method returns a value of true or false to indicate whether a matching contact exists. You can check for duplicates by first name, last name, Tax ID and birth date.

The CheckDuplicateContact method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description
LastName	String	Yes	Last name
FirstName	String	Yes	First name
TaxID	String	No	Tax ID/Social Security Number
Dob	String	No	Birth date

Example

This request checks for a duplicate record for a contact named Genevieve Bates:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <CheckDuplicateContact>
      <LastName>Bates</LastName>
      <FirstName>Genevieve</FirstName>
      <TaxID>111111111</TaxID>
      <Dob>09-08-1968</Dob>
    </CheckDuplicateContact>
  </method>
</request>
```

The response indicates that a duplicate was found:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <CheckDuplicateContact>true</CheckDuplicateContact>
  </method>
  <_status>OK</_status>
  <_systime>2015-06-09T18:30:22</_systime>
  <!--Cost 25 mill seconds.-->
</response>
```

CreateSVAdvisorUser Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The CreateSVAdvisorUser method creates a SmartView for Advisors user account for a specified advisor contact.

The CreateSVAdvisorUser method is invoked using the method operation.

The advisor contact must exist in the database before this method can be used. If the advisor contact does not exist, create it first using the insert operation.

Parameters

Parameter Name	Type	Required?	Description
AgentID	String	Yes	This is the object ID of the advisor contact record.
LoginName	String	Yes	This is the user name that the advisor will use to sign in to SmartOffice. The user name must be unique in the office.
TaxID	String	No	This is the value passed in NameID when SAML SSO is used.
AdvisorVUType	String	No	This specifies the format of the SmartView for Advisors user account being created. Valid values are 0 (SmartView for Advisors) or 1 (SmartView

			Case Status). If this parameter is not specified, it defaults to 0.

Example

First, use the insert operation to create the advisor contact if it does not exist. At minimum, specify the advisor's last name, first name and e-mail address. You will also specify the client type (7) for advisor contacts:

```
<request version='1.0'>
    <header>...</header>
    <insert>
        <Agent>
            <Contact>
                <LastName>Hannes</LastName>
                <FirstName>Walter</FirstName>
                <ClientType>7</ClientType>
                <WebAddresses>
                    <WebAddress>
                        <Address>walter.hannes@example.com</Address>
                        <WebAddressType>1</WebAddressType>
                    </WebAddress>
                </WebAddresses>
            </Contact>
        </Agent>
    </insert>
</request>
```

The response confirms that the advisor record was created:

```
<response version="1.0">
    <header>
        <sessionClosed/>
    </header>
    <insert>
        <Agent id="Agent.1.210" _type="obj" _status="inserted">
            <Contact id="Contact.1.211" _type="obj" _status="inserted">
                <WebAddresses _type="objs">
                    <WebAddress id="WebAddress.1.93" _type="obj"
_status="inserted"/>
                </WebAddresses>
            </Contact>
        </Agent>
    </insert>
    <_status>OK</_status>
</response>
```

Next, capture the Agent ID from the response for use in the next request, which creates the SmartView for Advisors user account:

```
<request version='1.0'>
    <header>...</header>
    <method>
        <CreateSVAdvisorUser>
            <AgentID>Agent.1.210</AgentID>
            <LoginName>walter.hannes</LoginName>
            <ExtLink>EVAL6_SDC_Walter.Hannes</ExtLink>
            <AdvisorVUType>1</AdvisorVUType>
        </CreateSVAdvisorUser>
    </method>
</request>
```

The response indicates the the SmartView for Advisors account was created successfully and returns the object ID of the new user account:

```
<response version="1.0">
    <header>
        <sessionClosed />
    </header>
    <method>
        <CreateSVAdvisorUser>
            <User id="User.1.9" _type="obj" _status="inserted" />
        </CreateSVAdvisorUser>
    </method>
    <_status>OK</_status>
    <_systime>2017-03-13T23:58:29</_systime>
    <!-Cost 1083 mill seconds.-->
</response>
```

DelContact Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The DelContact method removes a contact from a set.

The DelContact method is invoked using the method operation.

Note that this method deletes only the contact's set membership, not the contact itself.

Parameters

Parameter Name	Type	Required?	Description
ContactID	String	Yes	This is the object ID of the contact that will be removed from the set.
SetOfficeID	String	Yes	Use the office identifier only, not the full object ID (e.g., 1 instead of Office.1).
SetID	String	Yes	Use the set identifier only, not the full object ID (e.g., 20 instead of Set.1.20).

Example

The following example removes a contact with object ID Contact.1.14 from a set:

```
<request version='1.0'>
    <header>...</header>
    <method>
        <DelContact>
            <ContactID>Contact.1.14</ContactID>
            <SetOfficeID>1</SetOfficeID>
            <SetID>20</SetID>
        </DelContact>
    </method>
</request>
```

The response confirms the deletion:

```
<response version="1.0">
    <header>
        <sessionClosed/>
    </header>
    <method>
        <DelContact>
            <DeletionResult Contact="1"/>
        </DelContact>
    </method>
    <_status>OK</_status>
    <_systime>2015-10-12T18:05:24</_systime>
    <!--Cost 26 mill seconds.-->
</response>
```

DelContactSubObjs Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The DelContactSubObjs method deletes all phone number, address and e-mail/web address entries from a contact record.

The DelContactSubObjs method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description
ContactID	String	Yes	This is the object ID of the contact record.

Example

The following example deletes all phone numbers, addresses and e-mail/web addresses from a contact:

```
<request version='1.0'>
    <header>...</header>
    <method>
        <DelContactSubObjs>
            <ContactID>Contact.1.18</ContactID>
        </DelContactSubObjs>
    </method>
</request>
```

The response displays a count of the number of each record type deleted:

```
<response version="1.0">
    <header>
        <sessionClosed/>
    </header>
    <method>
        <DelContactSubObjs>
            <DeletionResult Addresses="2" Phones="2" Webaddresses="0"/>
        </DelContactSubObjs>
    </method>
    <_status>OK</_status>
    <_systime>2015-10-12T18:32:35</_systime>
    <!--Cost 69 mill seconds.-->
</response>
```

DeleteActivityContact Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The DeleteActivityContact removes a contact from a calendar activity.

The DeleteActivityContact method is invoked using the method operation.

Note that an activity's primary contact cannot be removed using this method.

Parameters

Parameter Name	Type	Required?	Description
ActivityID	String	Yes	This is the object ID of the calendar activity.

ContactID	String	Yes	This is the object ID of the contact you want to delete from the activity.

Example

This example request deletes a contact with object ID Contact.1.130 from an activity with object ID Activity.1.11:

```
<request version='1.0'>
    <header>...</header>
    <method>
        <DeleteActivityContact>
            <ActivityID>Activity.1.11</ActivityID>
            <ContactID>Contact.1.130</ContactID>
        </DeleteActivityContact>
    </method>
</request>
```

The response confirms the deletion:

```
<response version="1.0">
    <header>
        <sessionClosed/>
    </header>
    <method>
        <DeleteActivityContact>
            <ActivityContact _type="obj" _status="deleted"/>
        </DeleteActivityContact>
    </method>
    <_status>OK</_status>
    <_systime>2015-10-12T19:01:06</_systime>
    <!--Cost 23 mill seconds.-->
</response>
```

DisableUser Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The DisableUser method changes the status of a SmartOffice user account to Disabled.

The DisableUser method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description
ExternalID	String	Yes	The value of this parameter is formatted as servername_officename_username (e.g., MYSO_myoffice_daniel).

Example

The following example request disables a user account named daniel in the office MYOFFICE on the server MYSO:

```
<request version='1.0'>
    <header>...</header>
    <method>
        <DisableUser>
            <ExternalID>MYSO_MYOFFICE_daniel</ExternalID>
        </DisableUser>
    </method>
</request>
```

The response confirms the user account's change of status:

```
<response version="1.0">
    <header>
        <sessionClosed/>
    </header>
    <method>
        <DisableUser>1</DisableUser>
    </method>
    <_status>OK</_status>
    <_systime>2015-10-12T21:00:08</_systime>
    <!--Cost 29 mill seconds.-->
</response>
```

DynamicReport Method

In this Topic Hide

- Overview
- Parameters
- Examples
  - Basic Dynamic Report Request
  - Customizing the Report Filter Definition

Overview

The `DynamicReport` method retrieves the current data for a Dynamic Report by report name. The method also allows for the filter on the report to be redefined at run-time.

The `DynamicReport` method is invoked using the `method` operation.

Parameters

Parameter Name	Type	Required?	Description
`reportName`	String	Yes	This is the name of the Dynamic Report.
`format`	Integer	Yes	This parameter indicates the data format in which report results are returned. Valid values are 1 for comma-separated values (CSV) and 2 for XML.
`pageNo`	Integer	Yes	This parameter identifies the "page" of the report being retrieved. For the initial request, the value is typically 0.

			If you send subsequent requests to retrieve additional pages of report results, increment this value by one in each new request.
`pageSize`	Integer	Yes	This is the size of a report "page," i.e., the number of records to return in a single page. The maximum value allowed is 2000.
`searchKey`	String	No	This parameter is required if you intend to submit multiple requests for additional pages of report results. In the initial request, this value is -1. In all subsequent requests to retrieve additional pages of data, this value must be the same as the `searchid` parameter returned in the response to the initial request.
`runtimefilter`	String	No	This parameter is required only if you want to modify the default report filter definition. Specify the modified filter syntax using this parameter. The default filter syntax can be retrieved using the `DynamicReportFilter` method. See the Examples section below for details.

Examples

Basic Dynamic Report Request

The following example requests the results of a Dynamic Report named "Clients with Children" using the default report filter definition.

Note the `keepsession` element in the request header; if you intend to send subsequent requests to retrieve more "pages" of report results, this element is required and must be set to `true`.

```
<request version='1.0'>
    <header>
        <office/>
        <user/>
        <password/>
        <keepsession>true</keepsession>
    </header>
    <method>
        <DynamicReport>
            <reportName>Clients with Children</reportName>
            <format>1</format>
            <searchKey>-1</searchKey>
            <pageNo>0</pageNo>
            <pageSize>10</pageSize>
        </DynamicReport>
    </method>
</request>
```

The response returns the data for the report enclosed in the `export` element, with the report data itself enclosed in the `data` element:

```
<response version="1.0">
    <header/>
    <method>
        <DynamicReport>
            <export total="10" searchid="1" pagesize="11" more="true">
                <data>Contact Name,# of Child,Record Type,,"First Name, Last
Name",,Source,Subsystem Type
                "Gibbs, Tracy",2,Contact,Identity,Tracy Gibbs,0,,Contact
                "Chambers, Essie",2,Contact,Identity,Essie
Chambers,0,,Contact
                "Lambert, Lillian",2,Contact,Identity,Lillian
Lambert,0,,Contact
                "Lewis, Tara",2,Contact,Identity,Tara Lewis,0,,Contact
                "Walsh, Homer",2,Contact,Identity,Homer Walsh,0,,Contact
                "Gill, Wilbur",2,Contact,Identity,Wilbur Gill,0,,Contact
                "Morris, George",2,Contact,Identity,George Morris,0,,Contact
                "Hodges, Harold",2,Contact,Identity,Harold Hodges,0,,Contact
                "Fleming, Catherine",2,Contact,Identity,Catherine
Fleming,0,,Contact
                "Beck, Robyn",2,Contact,Identity,Robyn Beck,0,,Contact</data>
            </export>
        </DynamicReport>
    </method>
    <_status>OK</_status>
    <_systime>2016-04-19T01:04:31</_systime>
    <!--Cost 609 mill seconds.-->
</response>
```

Note the following in the response:

- The `more` attribute indicates whether additional pages of results are available.
- The `searchid` attribute allows you to retrieve additional pages of report results (see the next example).

To retrieve the next page of results, send a new request that includes the `searchid` attribute from the initial response as the value of the `searchKey` parameter. In addition, increment the `pageNo` parameter by 1:

```
<request version='1.0'>
    <header>
        <office/>
        <user/>
        <password/>
        <keepsession>true</keepsession>
    </header>
    <method>
        <DynamicReport>
            <reportName>Clients with Children</reportName>
            <format>1</format>
            <searchKey>1</searchKey>
            <pageNo>1</pageNo>
            <pageSize>10</pageSize>
        </DynamicReport>
    </method>
</request>
```

In the response, the value of the more attribute is `false` to indicate that no more results are available.

```
<response version="1.0">
    <header/>
    <method>
        <DynamicReport>
            <export total="4" searchid="1" pagesize="10" more="false">
                <data>Contact Name,# of Child,Record Type,,"First Name, Last
Name",,Source,Subsystem Type
                "Ward, Marcella",2,Contact,Identity,Marcella Ward,0,,Contact
                "Moody, Lynda",2,Contact,Identity,Lynda Moody,0,,Contact
                "Burns, Pearl",2,Contact,Identity,Pearl Burns,0,,Contact
                "Jacobs, Derrick",2,Contact,Identity,Derrick
Jacobs,0,,Contact</data>
            </export>
        </DynamicReport>
    </method>
    <_status>OK</_status>
    <_systime>2016-04-19T16:48:15</_systime>
    <!--Cost 75 mill seconds.-->
```

```
</response>
```

If more results were available, you could then send a third request in the same manner, keeping the `searchKey` parameter the same (in this case, 1) and incrementing the `pageNo` parameter to 2.

Customizing the Report Filter Definition

You can use the `DynamicReportFilter` method together with the `DynamicReport` method to change the report's default filter definition.

First, send a request using the `DynamicReportFilter` method to retrieve the report's default filter:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <DynamicReportFilter>
      <reportName>A - Clients By City</reportName>
    </DynamicReportFilter>
  </method>
</request>
```

The response returns the filter definition syntax:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <DynamicReportFilter>
<![CDATA[
  <filter fltrkey="1">
    <filter fltrkey="2">
      <systemFilter fltrkey="3" fltrnm="Postal Address" id="address-contact"
sqlop="eq">
        <filter fltrkey="2">
          <element colName="City" colid="393223" fltrkey="3" sqlop="eq"
value="Los Angeles"/>
        </filter>
      </systemFilter>
    </filter>
    <filter fltrkey="3">
      <element colName="Type" colid="65627" fltrkey="5" sqlop="eq"
value="Client"/>
      <element colName="Sub-Type" colid="65541" fltrkey="6" sqlop="eq"
value="A"/>
    </filter>
  </filter>
]]>
    </DynamicReportFilter>
  </method>
```

```
  <_status>OK</_status>
  <!--Cost 49 mill seconds.-->
</response>
```

Next, modify the filter syntax and include it in the `runtimefilter` parameter when using the `DynamicReport` method. In this example, the syntax has been modified to filter for "Portland" instead of the default "Los Angeles" in the City column:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <DynamicReport>
      <reportName>A - Clients By City</reportName>
      <format>2</format>
      <searchKey>-1</searchKey>
      <pageNo>-1</pageNo>
      <pageSize>0</pageSize>
      <runtimefilter>
<![CDATA[
  <filter fltrkey="1">
    <filter fltrkey="2">
      <systemFilter fltrkey="3" fltrnm="Postal Address" id="address-contact"
sqlop="eq">
        <filter fltrkey="2">
          <element colName="City" colid="393223" fltrkey="3" sqlop="eq"
value="Portland"/>
        </filter>
      </systemFilter>
    </filter>
    <filter fltrkey="3">
      <element colName="Type" colid="65627" fltrkey="5" sqlop="eq"
value="Client"/>
      <element colName="Sub-Type" colid="65541" fltrkey="6" sqlop="eq"
value="A"/>
    </filter>
  </filter>
]]>
      </runtimefilter>
    </DynamicReport>
  </method>
</request>
```

The response returns the appropriate report data based on the modified filter:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <DynamicReport>
      <export total="1" searchid="-1" pagesize="2001" more="false">
```

```
        <data>
          <DynamicReport>
            <heading>
              <col id="0">Contact Name</col>
              <col id="1">Type</col>
              <col id="2">Source</col>
              <col id="3">Occupation</col>
              <col id="4">Birth Date</col>
              <col id="5">Review Date</col>
              <col id="6">Total Premium</col>
            </heading>
            <rows>
              <row>
                <col id="0">Ackerman, Joseph</col>
                <col id="1">Client</col>
                <col id="6">0</col>
              </row>
            </rows>
          </DynamicReport>
        </data>
      </export>
    </DynamicReport>
  </method>
  <_status>OK</_status>
  <!--Cost 151 mill seconds.-->
</response>
```


DynamicReportFilter Method

In this Topic Hide

- Overview
- Parameters
- Example
- Popular Date Range Values

Overview

The DynamicReportFilter method retrieves the default filter syntax of a Dynamic Report. The filter syntax can be used as part of a DynamicReport method request.

The DynamicReportFilter method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description
reportName	String	Yes	This is the name of the Dynamic Report.

Example

The following example retrieves the filter for the Dynamic Report named A - Clients By City.:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <DynamicReportFilter>
      <reportName>A - Clients By City</reportName>
    </DynamicReportFilter>
  </method>
</request>
```

The response returns the filter syntax:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <DynamicReportFilter>
<![CDATA[
  <filter fltrkey="1">
    <filter fltrkey="2">
      <systemFilter fltrkey="3" fltrnm="Postal Address" id="address-contact"
sqlop="eq">
        <filter fltrkey="2">
          <element colName="City" colid="393223" fltrkey="3" sqlop="eq"
value="Los Angeles"/>
        </filter>
      </systemFilter>
    </filter>
    <filter fltrkey="3">
      <element colName="Type" colid="65627" fltrkey="5" sqlop="eq"
value="Client"/>
      <element colName="Sub-Type" colid="65541" fltrkey="6" sqlop="eq"
value="A"/>
    </filter>
```

```
  </filter>
]]>
    </DynamicReportFilter>
  </method>
  <_status>OK</_status>
  <!--Cost 49 mill seconds.-->
</response>
```

Popular Date Range Values

When modifying a filter in which the sqlop attribute is set to popular dates, refer to the following table for the value you need to enter.

Date Range	Value
Current	
Today	32
Current Week	27
Current Month	0
Current Quarter	2
Current Year	1
Previous	
Yesterday	30
Previous 7 Days	22
Previous 15 Days	33
Previous 30 Days	21
Previous 60 Days	34
Previous 90 Days	88
Previous Week	25
Previous 2 Weeks	35

Previous Month	3
Previous 2 Months	36
Previous 3 Months	6
Previous 6 Months	7
Previous 9 Months	8
Previous 12 Months	9
Previous Quarter	5
Previous Year	4
Previous 2 Years	37
Previous 180 Days	74
Previous 360 Days	75
All Days in Past	28
All Days in Past + Today	150
Next	
Tomorrow	31
Next 7 Days	24
Next 15 Days	38
Next 30 Days	23
Next 60 Days	39
Next Week	26
Next 2 Weeks	40
Next Month	14
Next 2 Months	41
Next 3 Months	17

Next 6 Months	18
Next 9 Months	19
Next 12 Months	20
Next Quarter	16
Next Year	15
Next 2 Years	42
Next 180 Days	76
Next 360 Days	77
All Days in Future	95
All Days in Future + Today	29
Before	
Before 7 Days	54
Before 15 Days	55
Before 30 Days	56
Before 45 Days	72
Before 60 Days	57
Before 2 Weeks	58
Before 1 Month	48
Before 2 Months	49
Before 3 Months	50
Before 6 Months	51
Before 9 Months	52
Before 12 Months	53
Before 1 Year	46

Before 2 Years	47
After	
Beginning in 7 Days	67
Beginning in 15 Days	68
Beginning in 30 Days	69
Beginning in 45 Days	73
Beginning in 60 Days	70
Beginning in 2 Weeks	71
Beginning in 1 Month	61
Beginning in 2 Months	62
Beginning in 3 Months	63
Beginning in 6 Months	64
Beginning in 9 Months	65
Beginning in 12 Months	66
Beginning in 2 Years	60
Quarter-Current/Previous Year (Based on Current Date)	
1st Quarter	10
2nd Quarter	11
3rd Quarter	12
4th Quarter	13
Quarter-Previous Year/2 Years Ago (Based on Current Date)	
1st Quarter	78
2nd Quarter	79

3rd Quarter	80
4th Quarter	81
Quarter - Current Year	
1st Quarter	142
2nd Quarter	143
3rd Quarter	144
4th Quarter	145
Quarter - Previous Year	
1st Quarter	146
2nd Quarter	147
3rd Quarter	148
4th Quarter	149
To Date	
Week-to-Date	82
Month-to-Date	43
Quarter-to-Date	44
Year-to-Date	45

DynamicReportFilterDetail Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The DynamicReportFilterDetail method returns the detailed default filter syntax of a Dynamic Report.

The DynamicReportFilterDetail method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description
reportId	String	Yes	This is the object ID of the Dynamic Report.

Example

The following example retrieves the detailed filter syntax for a report named "Clients With Children" whose object ID is DynReport.1.3:

```
<request version='1.0'>
    <header>...</password>
    </header>
    <method>
        <DynamicReportFilterDetail>
            <reportId>DynReport.1.3</reportId>
        </DynamicReportFilterDetail>
    </method>
</request>
```

The response returns the filter details:

```
<?xml version="1.0" encoding="UTF-8" ?>
<response version="1.0">
    <header>
        <sessionClosed />
    </header>
    <method>
        <DynamicReportFilterDetail>
            <![CDATA[
              <filter fltrkey="4">
                <element colName="Type" colid="65627" fltrkey="5"
lookupId="5" sqldesc="Equal to" sqlop="eq" tableName="Contact" type="dtStr"
uiValue="Client" value="Client"/>
```

```
                <element colName="# of Child" colid="589830" fltrkey="6"
sqldesc="Greater Than" sqlop="gt" tableName="Personal" type="dtShort"
uiValue="0" value="0"/>
              </filter>
            ]]>
        </DynamicReportFilterDetail>
    </method>
    <_status>OK</_status>
    <_systime>2016-04-28T19:23:34</_systime>
    <!-- Cost 21 mill seconds. -->
</response>
```

GetActColors Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetActColors method returns a list of the colors used to help identify activity types and subtypes in the SmartOffice calendar. Colors are returned in hexadecimal format.

The GetActColors method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description
Type	String	Yes	Valid values are 1 (activity type) or 2 (activity subtype).

Example

The following example request retrieves the colors for activity types:

```
<request version='1.0'>
    <header>...</header>
    <method>
        <GetActColors>
            <Type>1</Type>
        </GetActColors>
    </method>
</request>
```

In the response, note that the ActType IDs correspond to the values found in the ActivityType and ActivitySubtype lookup tables in the API Dictionary:

```
<?xml version="1.0" encoding="UTF-8" ?>
<response version="1.0">
    <header>
        <sessionClosed />
    </header>
    <method>
        <GetActColors>
            <ActColors>
                <ActType id="1">
                    <Color>#FFC500</Color>
                </ActType>
                <ActType id="2">
                    <Color>#0012BB</Color>
                </ActType>
                <ActType id="3">
                    <Color>#4AB62E</Color>
                </ActType>
                <ActType id="4">
                    <Color>#000000</Color>
                </ActType>
                <ActType id="5">
                    <Color>#7D7D7D</Color>
                </ActType>
                <ActType id="6">
                    <Color>#BB0004</Color>
                </ActType>
                <ActType id="7">
                    <Color>#c6c6c6</Color>
                </ActType>
            </ActColors>
        </GetActColors>
    </method>
    <_status>OK</_status>
    <_systime>2020-10-01T17:32:46</_systime>
    <!-- Cost 67 mill seconds. -->
</response>
```

GetAdvisorCommission Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetAdvisorCommission method retrieves all payable commission records linked to a specified SmartView for Advisors user account.

The GetAdvisorCommission method is invoked using the method operation.

This method returns a variety of information about each payable commission record, including the policy number, the advisor's role on the policy, the receivable amount, the payable due date, the paid amount, the status of the payable commission transaction and the commission type.

Parameters

Parameter Name	Type	Required?	Description
UserID	String	Yes	This is the object ID of the advisor's SmartView for Advisors user account.

Example

The following example request retrieves the payable commission records for a SmartView for Advisor user account with object ID User.1.16:

```
<request version='1.0'>
    <header>...</header>
    <method>
        <GetAdvisorCommission>
            <UserID>User.1.16</UserID>
        </GetAdvisorCommission>
```

```
    </method>
</request>
```

In this example, the response returns details of one commission transaction:

```
<response version="1.0">
    <header/>
    <method>
        <GetAdvisorCommission>
            <Contact id="Contact.1.112" _type="obj">
                <FirstName>Phil</FirstName>
                <LastName>Anderson</LastName>
                <CommPayables _type="objs">
                    <CommPayable id="CommPayable.1.11" _type="obj">
                        <CurrentRole>Primary Advisor</CurrentRole>
                        <PolicyNo>4323452</PolicyNo>
                        <Receivable>50</Receivable>
                        <PayableDueDate>2015-10-28</PayableDueDate>
                        <PaidAmt>0</PaidAmt>
                        <Status>Open</Status>
                        <CommType>Base</CommType>
                        <ComponentPrem>0</ComponentPrem>
                        <ReceivablePerc>50</ReceivablePerc>
                        <ReceivablePercOf>Premium</ReceivablePercOf>
                    </CommPayable>
                </CommPayables>
            </Contact>
        </GetAdvisorCommission>
    </method>
    <_status>OK</_status>
    <_systime>2015-10-20T22:00:07</_systime>
    <!--Cost 19 mill seconds.-->
</response>
```

GetConfiguration Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetConfiguration method retrieves the content of an element in the SmartOffice server configuration file (config.xml).

The GetConfiguration method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description
Key	String	Yes	This is the element in the config.xml file whose content you want to retrieve.

Example

The following example retrieves the content of the lockAfterLoginFailures element, which is a child of the passwordOptions element:

```
<request version='1.0'>
    <header>...</header>
    <method>
        <GetConfiguration>
            <Key>passwordOptions.lockAfterLoginFailures</Key>
        </GetConfiguration>
    </method>
</request>
```

The response returns the content of the element:

```
<response version="1.0">
    <header>
        <sessionClosed/>
    </header>
    <method>
        <GetConfiguration>
            <passwordOptions.lockAfterLoginFailures>3</passwordOptions.lockAf
terLoginFailures>
        </GetConfiguration>
    </method>
    <_status>OK</_status>
    <_systime>2015-10-20T22:31:00</_systime>
    <!--Cost 25 mill seconds.-->
</response>
```

GetContactDOB Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetContactDOB method returns all contacts whose birth dates fall within a specified date range and who are assigned to a particular SmartOffice user. The response includes each contact's first name, last name, contact ID and birth date.

The GetContactDOB method is invoked as part of the method operation.

Parameters

Parameter Name	Type	Required?	Description
UserID	String	Yes	This is the object ID of the user account assigned to contacts.
FromDate	String	Yes	This is the first date in the date range. Use the format year-month-day. Example: 1950-01-01.
ToDate	String	Yes	This is the last date in the date range. Use the format year-month-day. Example: 1960-01-01.

Example

The following example request retrieves all contacts born between January 1, 1950, and December 31, 1955, to which the user with object ID User.1.2 has been assigned:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <GetContactDOB>
      <UserID>User.1.2</UserID>
      <FromDate>1950-01-01</FromDate>
      <ToDate>1955-12-31</ToDate>
    </GetContactDOB>
  </method>
</request>
```

The response includes each contact's name, date of birth and object ID:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <GetContactDOB>
      <Contact id="Contact.1.12" _type="obj">
        <FirstName>Paulette</FirstName>
        <LastName>Garner</LastName>
        <Person id="Person.1.12" _type="obj">
          <Dob>1952-07-03</Dob>
        </Person>
      </Contact>
      <Contact id="Contact.1.14" _type="obj">
        <FirstName>Genevieve</FirstName>
        <LastName>Bates</LastName>
        <Person id="Person.1.14" _type="obj">
          <Dob>1954-10-16</Dob>
        </Person>
      </Contact>
    </GetContactDOB>
  </method>
  <_status>OK</_status>
  <_systime>2015-06-04T22:38:49</_systime>
  <!--Cost 36 mill seconds.-->
</response>
```

GetContactsForEmail Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetContactsForEmail method returns all contacts in SmartOffice who have one or more specified e-mail addresses.

The GetContactsForEmail method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description
EmailAddresses	String	Yes	This parameter accepts one or more e-mail addresses. Use commas to separate multiple addresses.

Example

The following example request retrieves information about all contacts with the e-mail address jackerman@example.com:

```
<request version='1.0'>
    <header>...</header>
    <method>
        <GetContactsForEmail>
            <EmailAddresses>jackerman@example.com</EmailAddresses>
        </GetContactsForEmail>
    </method>
</request>
```

The response includes each contact's full name, object ID, contact type, preferred address, preferred phone number, e-mail/web addresses (including preferred e-mail address), employer, occupation, job title and more:


```xml
<response version="1.0">
   <header>
      <sessionClosed />
   </header>
   <method>
      <GetContactsForEmail>
         <EZMail _type="obj" />
         <WebAddress _type="obj">
            <Address>jackerman@example.com</Address>
            <Contact _type="obj" id="Contact.1.1">
               <FullName>Joseph Ackerman</FullName>
               <ContactType>1</ContactType>
               <Greeting>Joseph</Greeting>
               <PreferredAddress _type="obj" id="Address.1.1">
                  <Line1>555 Main St.</Line1>
                  <Line2>2nd Floor</Line2>
                  <Line3>Suite 30</Line3>
                  <City>Torrance</City>
                  <State>CA</State>
               </PreferredAddress>
               <PreferredPhone _type="obj" id="Phone.1.1">
                  <PhoneNumber>(123) 555-2312 # 555</PhoneNumber>
               </PreferredPhone>
               <PreferredEmailAddress _type="objs">
                  <WebAddress _type="obj" id="WebAddress.1.1">
                     <Address>jackerman@example.com</Address>
                  </WebAddress>
               </PreferredEmailAddress>
               <WebAddresses _type="objs">
                  <WebAddress _type="obj" id="WebAddress.1.234">
                     <Address>jackerman@example.com</Address>
                  </WebAddress>
                  <WebAddress _type="obj" id="WebAddress.1.235">
                     <Address>jackerman</Address>
                  </WebAddress>
                  <WebAddress _type="obj" id="WebAddress.1.321">
                     <Address>http://jackerman.example.com</Address>
                  </WebAddress>
               </WebAddresses>
            </Contact>
         </WebAddress>
      </GetContactsForEmail>
   </method>
   <_status>OK</_status>
   <_systime>2020-05-28T16:18:30</_systime>
   <!-- Cost 169 mill seconds. -->
</response>
```

GetCustomLabelDetail Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetCustomLabelDetail method returns the label assigned to a particular custom field for a particular SmartOffice module and record type.

The GetCustomLabelDetail method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description
SubModuleId	Integer	Yes	Identifies the record type for which you want to retrieve a list of custom field labels. For a list of these IDs, refer to the topic about the GetCustomLabels method.
CustomLabelName	String	Yes	This is the ID of the office in which the SmartView for Advisors user account resides. Note that this value is the primary key (e.g., 1), not the full object ID (e.g., Office.1).

Example

The following request retrieves the label assigned to the Lookup1 custom field for the individual contact record type (SubModuleId=65539):

```
<request version='1.0'>
<header>...</header>
<method>
   <GetCustomLabelDetail>
      <SubModuleId>65539</SubModuleId>
      <CustomLabelName>Lookup1</CustomLabelName>
   </GetCustomLabelDetail>
</method>
</request>
```

The response returns the field label:

```
<response version="1.0">
   <header>
      <sessionClosed />
   </header>
   <method>
      <GetCustomLabelDetail>
         <CustomLabelDetail>
            <CustomLabel>Favorite Sport</CustomLabel>
         </CustomLabelDetail>
      </GetCustomLabelDetail>
   </method>
   <_status>OK</_status>
   <!-- Cost 708 mill seconds. -->
</response>
```

GetCustomLabels Method

In this Topic Hide

- Overview
- Parameters
- Reference
- Example

Overview

The GetCustomLabels method returns a list of all custom field labels for a particular SmartOffice module and record type.

The GetCustomLabels method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description
SubModuleId	Integer	Yes	This is the record type for which you want to retrieve a list of custom field labels. See the Reference section below for a list of valid values.
GroupID	Long Integer	Yes	This is the user group whose custom field labels you want to retrieve (in SmartOffice, different user groups can have their own custom fields). The most common value is 1, which refers to the default "All Users in the Office" user group.

Reference

The following table lists the values you can use for the SubModuleId parameter.

SmartOffice Module	Record Type	SubModuleId Value
Contact	Individual	65539
	Business	65561
Policy	All	1769495
	Life	1769483
	Auto	1769484
	Disability	1769485
	Annuity	1769486
	Long-Term Care	1769487

	Medical	1769488
	Homeowner	1769489
	Umbrella	1769490
	Other	1769491
	Property & Casualty	1769498
	Critical Illness	1769493
Investment Account	All	2621451
Investment Position	All	2686989
Pending Case	All	4587543
	Life	4587528
	Auto	4587532
	Disability	4587533
	Annuity	4587534
	Long-Term Care	4587535
	Medical	4587536
	Homeowner	4587537
	Umbrella	4587538
	Other	4587539
	Property & Casualty	4587545
	Critical Illness	4587541
Product	All	6750213
Advisor	All	8192005
Agency	All	8192021
Carrier	All	8716304

Candidate	All	9830407
Household	All	18087946
Opportunity	All	19857415
Leads	All	19922953
Group Plan	Life	43778051
	Dental	43778053
	Long-Term Care	43778054
	Medical	43778055
	Accidental Death & Dismemberment	43778056
	Retirement	43778057
	Short-Term Disability	43778058
	Travel	43778059
	Vision	43778060
	Section 125	43778061
	Long-Term Disability	43778062
	Other	43778063
	Cancer	43778065
	Hospital	43778067
Event	All	61734920
Firm	All	151257096
Producer	All	151650313
Rider	All	151650313

Example

The following request retrieves a list of custom fields for the individual contact record type (SubModuleId=65539):

```
<request version='1.0'>
<header>...</header>
<method>
   <GetCustomLabels>
      <SubModuleId>65539</SubModuleId>
      <GroupID>1</GroupID>
   </GetCustomLabels>
</method>
</request>
```

The response indicates that individual contact records in this office have three labeled custom fields:

```
<response version="1.0">
   <header>
      <sessionClosed />
   </header>
   <method>
      <GetCustomLabels>
         <CustomLabels>
            <CustomLabel>
               <CustomColumn>ALPHANUM1</CustomColumn>
               <LabelName>Primary Charity</LabelName>
            </CustomLabel>
            <CustomLabel>
               <CustomColumn>ALPHANUM2</CustomColumn>
               <LabelName>Secondary Charity</LabelName>
            </CustomLabel>
            <CustomLabel>
               <CustomColumn>DATE1</CustomColumn>
               <LabelName>Anniversary</LabelName>
            </CustomLabel>
         </CustomLabels>
      </GetCustomLabels>
   </method>
   <_status>OK</_status>
   <!-- Cost 708 mill seconds. -->
</response>
```

GetDeletedContacts Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetDeletedContacts method returns the object IDs of all contacts that exist in the Deleted Records area of SmartOffice.

The GetDeletedContacts method is invoked using the method operation.

Parameters

This method takes no parameters.

Example

Here is how the request is sent:

```
<request version='1.0'>
  <header>...</header>
    <method>
      <GetDeletedContacts/>
    </method>
</request>
```

The response returns the deleted contacts' object IDs:

```
<response version="1.0">
  <header>
    <sessionClosed />
  </header>
  <method>
    <GetDeletedContacts>
      <Contact id="Contact.1.88" _type="obj" />
      <Contact id="Contact.1.90" _type="obj" />
    </GetDeletedContacts>
  </method>
  <_status>OK</_status>
  <_systime>2016-04-28T22:34:08</_systime>
```

```
  <!-- Cost 16 mill seconds. -->
</response>
```

GetDeletedPolicies Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetDeletedPolicies method returns the object IDs of all insurance policies that exist in the Deleted Records area of SmartOffice.

The GetDeletedPolicies method is invoked using the method operation.

Parameters

This method takes no parameters.

Example

Here is how the request is sent:

```
<request version='1.0'>
  <header>...</header>
    <method>
      <GetDeletedPolicies/>
    </method>
</request>
```

The response returns the deleted policies' object IDs:

```
<response version="1.0">
  <header>
    <sessionClosed />
  </header>
  <method>
    <GetDeletedPolicies>
      <Policy _type="obj" id="Policy.1.2" />
```

```
      <Policy _type="obj" id="Policy.1.7" />
    </GetDeletedPolicies>
  </method>
  <_status>OK</_status>
  <_systime>2016-04-28T22:34:08</_systime>
  <!-- Cost 16 mill seconds. -->
</response>
```

GetHouseholdMember Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

Using a contact's object ID, you can retrieve a list of all members of the contact's household. If the household or contact does not exist, this method returns a status of OK.

The GetHouseholdMember method is invoked as part of the method operation.

Parameters

Parameter Name	Type	Required?	Description
ContactID	String	Yes	This is the object ID of the contact linked to the household.

Example

The following example request retrieves household member information for a contact with the object ID Contact.1.145:

```
<request version='1.0'>
```

```
  <header>...</header>
  <method>
    <GetHouseholdMember>
      <ContactID>Contact.1.145</ContactID>
    </GetHouseholdMember>
  </method>
</request>
```

The response includes the object IDs and names of each household member:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <GetHouseholdMember>
      <HouseHold id="HouseHold.1.292" _type="obj">
        <Name>Chen Family</Name>
        <HouseHoldMembers _type="objs">
          <HouseHoldMember id="HouseHoldMember.1.687" _type="obj">
            <Member>Head</Member>
            <Contact id="Contact.1.14561" _type="obj">
              <FirstName>Andrew</FirstName>
              <LastName>Chen</LastName>
            </Contact>
          </HouseHoldMember>
          <HouseHoldMember id="HouseHoldMember.1.688" _type="obj">
            <Member>Spouse</Member>
            <Contact id="Contact.1.14580" _type="obj">
              <FirstName>MJ</FirstName>
              <LastName>Chen</LastName>
            </Contact>
          </HouseHoldMember>
        </HouseHoldMembers>
      </HouseHold>
    </GetHouseholdMember>
  </method>
  <_status>OK</_status>
  <!--Cost 21 mill seconds.-->
</response>
```

GetInvtMaturityDate Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetInvtMaturityDate method retrieves all investment positions that will mature within a specified date range for all contacts assigned to a particular SmartOffice user.

The GetInvtMaturityDate method is invoked using the method operation.

This method returns the following data:

- Contact names
- Account types, names and numbers
- Position names, holding types, maturity dates and maturity values

Parameters

Parameter Name	Type	Required?	Description
UserID	String	Yes	This is the object ID of the user account.
FromDate	String	Yes	This is the starting date in the date range. Use the date format year-month-day (e.g., 2015-01-01).
ToDate	String	Yes	This is the last date in the date range. Use the date format year-month-day (e.g., 2015-01-01).

Example

The following example retrieves data about all positions maturing between Jan. 1, 2015, and December 31, 2015, for contacts assigned to the user whose object ID is User.1.2.:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <GetInvtMaturityDate>
      <UserID>User.1.2</UserID>
      <FromDate>2015-01-01</FromDate>
      <ToDate>2015-12-31</ToDate>
    </GetInvtMaturityDate>
  </method>
</request>
```

In this example response, one record is returned:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <GetInvtMaturityDate>
      <Contact id="Contact.1.72" _type="obj">
        <FirstName>Flora</FirstName>
        <LastName>Cannon</LastName>
        <AcctMasters _type="objs">
          <AcctMaster id="AcctMaster.1.11" _type="obj">
            <AcctNumber>34534534534</AcctNumber>
            <AcctName>Premier Brokerage</AcctName>
            <AcctType>Individual</AcctType>
            <Positions _type="objs">
              <Position id="Position.1.26" _type="obj">
                <InvestName>RESERVE PRIMARY FUND CL 25</InvestName>
                <HoldingType>Cash and Equivalents</HoldingType>
                <MaturityDate>2015-10-29</MaturityDate>
                <MaturityValue>1000</MaturityValue>
              </Position>
            </Positions>
          </AcctMaster>
        </AcctMasters>
      </Contact>
    </GetInvtMaturityDate>
  </method>
  <_status>OK</_status>
  <_systime>2015-10-20T23:03:05</_systime>
  <!--Cost 39 mill seconds.-->
</response>
```

GetOfficeMultiFactorAuthentication Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The `GetOfficeMultiFactorAuthentication` method indicates whether an office has two-step authentication enabled. The method returns a value of 0 (disabled) or 1 (enabled).

The `GetOfficeMultiFactorAuthentication` method is invoked using the `method` operation.

Note: To retrieve two-factor authentication status at the user account level, use the GetUserMultiFactorAuthentication method.

Parameters

Parameter Name	Type	Required?	Description
`OfficeID`	String	Yes	This is the object ID of the SmartOffice office.

Example

This example requests the two-step authentication status of an office with object ID Office.1:

```
<request version='1.0'>
  <header>...</header>
  <method>
```

```
    <GetOfficeMultiFactorAuthentication>
      <OfficeID>Office.1</OfficeID>
    </GetOfficeMultiFactorAuthentication>
  </method>
</request>
```

The response returns a status of 0 (disabled):

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <GetOfficeMultiFactorAuthentication>
      <OfficeMultiFactorAuthentication>
        <TwoWayAuthMode>0</TwoWayAuthMode>
      </OfficeMultiFactorAuthentication>
    </GetOfficeMultiFactorAuthentication>
  </method>
  <_status>OK</_status>
  <_systime>2017-09-27T22:45:07</_systime>
  <!-- Cost 16 mill seconds. -->
</response>
```

GetPolicyInvtData Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetPolicyInvtData method retrieves details of a specified contact's policy and investment holdings.

The GetPolicyInvtData method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description

ContactID	String	Yes	This is the object ID of the contact.

Example

This example request retrieves policy and investment holdings for a contact with object ID Contact.1.18:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <GetPolicyInvtData>
      <ContactID>Contact.1.18</ContactID>
    </GetPolicyInvtData>
  </method>
</request>
```

The response includes the contact's name and details about each holding:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <GetPolicyInvtData>
      <Contact id="Contact.1.18" _type="obj">
        <FirstName>Essie</FirstName>
        <LastName>Chambers</LastName>
        <Policies _type="objs">
          <Policy id="Policy.1.2" _type="obj">
            <CarrierName>Best Insurance</CarrierName>
            <HoldingType>Life</HoldingType>
            <PolicyNumber>267435</PolicyNumber>
            <AnnualPremium>0</AnnualPremium>
          </Policy>
        </Policies>
        <AcctMasters _type="objs">
          <AcctMaster id="AcctMaster.1.5" _type="obj">
            <AcctNumber>2345232354</AcctNumber>
            <AcctName>TD Ameritrade</AcctName>
            <AcctType>Individual</AcctType>
            <HowHeld>Directly Owned</HowHeld>
            <Owner>Client</Owner>
            <Status>Active/Open</Status>
            <Positions _type="objs">
              <Position id="Position.1.19" _type="obj">
                <InvestName>NVIDIA CORP COM</InvestName>
                <HoldingType>Stocks</HoldingType>
                <PurchaseDate>2016-03-13</PurchaseDate>
                <MaturityValue>0</MaturityValue>
                <TaxDeferred>No</TaxDeferred>
              </Position>
```

```
            </Positions>
          </AcctMaster>
        </AcctMasters>
      </Contact>
    </GetPolicyInvtData>
  </method>
  <_status>OK</_status>
  <_systime>2017-03-13T22:36:45</_systime>
  <!-Cost 31 mill seconds.   -->
</response>
```

GetSiteName Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetSiteName method returns the site name of the SmartOffice server receiving the request. In SmartOffice Pro, the site name appears as the Corporate Key field on the Basic Info content link.

The GetSiteName method is invoked using the method operation.

Parameters

This method takes no parameters.

Example

The request looks like this:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <GetSiteName/>
  </method>
</request>
```

The response returns the site name:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <GetSiteName>
      <Site>
        <Name>RGQA3CLD</Name>
      </Site>
    </GetSiteName>
  </method>
  <_status>OK</_status>
  <_systime>2018-03-13T00:18:21</_systime>
  <!-- Cost 19 mill seconds. -->
</response>
```

GetSystemTime Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetSystemTime method returns the current date and time on the SmartOffice server.

The GetSystemTime method is invoked using the method operation.

Parameters

This method takes no parameters.

Example

The request is as follows:

```
<request version='1.0'>
```

```
  <header>...</header>
  <method>
    <GetSystemTime/>
  </method>
</request>
```

The response includes the server date and time:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <GetSystemTime>2015-06-12T18:20:40</GetSystemTime>
  </method>
  <_status>OK</_status>
  <_systime>2015-06-12T18:20:40</_systime>
  <!--Cost 24 mill seconds.-->
</response>
```

GetUnassignedAccounts Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetUnassignedAccounts method returns the object IDs of all investment accounts in an office that have a particular account number and are not assigned to a contact.

The GetUnassignedAccounts method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description

OfficeID	Long	Yes	The object ID of the SmartOffice office.
AcctNumber	String	Yes	The investment account number.

Example

The following example request retrieves all investment accounts with account number 222222222 in an office with ID 1111111 that are not assigned to a contact:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <GetUnassignedAccounts>
      <OfficeID>1111111</OfficeID>
      <AcctNumber>222222222</AcctNumber>
    </GetUnassignedAccounts>
  </method>
</request>
```

The response returns the object IDs of the investment accounts:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <GetUnassignedAccounts>
      <AcctMaster _type="obj" id="AcctMaster.1111111.23"/>
      <AcctMaster _type="obj" id="AcctMaster.1111111.24"/>
      <AcctMaster _type="obj" id="AcctMaster.1111111.25"/>
    </GetUnassignedAccounts>
  </method>
  <_status>OK</_status>
  <_systime>2022-02-08T18:23:52</_systime>
  <!-- Cost 33 mill seconds. -->
</response>
```

GetUserLicense Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The GetUserLicense method returns the names and IDs of licenses enabled for a particular SmartOffice user account.

The GetUserLicense method is invoked using the method operation.

Parameters

Parameter Name	Type	Required?	Description
UserID	String	Yes	This is the object ID of the SmartOffice user account.

Example

The following example request retrieves the licenses for the user account with object ID User.1.2:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <GetUserLicense>
      <UserID>User.1.2</UserID>
    </GetUserLicense>
  </method>
</request>
```

The response returns the list of licenses:

```
<response version="1.0">
```

```xml
  <header>
    <sessionClosed/>
  </header>
  <method>
    <GetUserLicense>
      <User id="User.1.2" _type="obj">
        <Licenses _type="objs">
          <License id="License.1.2.48" _type="obj">
            <Name>extra.fields</Name>
          </License>
          <License id="License.1.2.44" _type="obj">
            <Name>CTMCarrierInterface</Name>
          </License>
          <License id="License.1.2.56" _type="obj">
            <Name>layoutcust.admin</Name>
          </License>
          <License id="License.1.2.22" _type="obj">
            <Name>policy</Name>
          </License>
          <License id="License.1.2.23" _type="obj">
            <Name>event.managment</Name>
          </License>
          <License id="License.1.2.26" _type="obj">
            <Name>agency.manager</Name>
          </License>
          <License id="License.1.2.16000" _type="obj">
            <Name>Named.User</Name>
          </License>
        </Licenses>
      </User>
    </GetUserLicense>
  </method>
  <_status>OK</_status>
  <_systime>2015-06-12T17:48:55</_systime>
  <!--Cost 145 mill seconds.-->
</response>
```


GetUserMultiFactorAuthentication Method

In this Topic Hide

- Overview
- Parameters
- Reference
- Example

Overview

The `GetUserMultiFactorAuthentication` method indicates whether a user account has two-step authentication enabled and returns the notification method that the user has selected. The status is indicated by a value of 0 (disabled) or 1 (enabled).

The `GetUserMultiFactorAuthentication` method is invoked using the `method` operation.

Note: To retrieve two-factor authentication status at the office account level, use the GetOfficeMultiFactorAuthentication method.

Parameters

Parameter Name	Type	Required?	Description
`UserID`	String	Yes	This is the object ID of the SmartOffice user account.

Reference

The response contains the following elements:

Element	Description	Possible Values
`TwoWayAuthMode`	Specifies whether two-step authentication is enabled.	0 (disabled) or 1 (enabled)
`OTPNotifyMethod`	Specifies the notification method the user has selected for receiving verification codes. If the user has disabled two-step authentication, this value reflects the	0 (none), 1 (text), 2 (e-mail), or 3 (text and e-mail)

	notification method selected the last time the user enabled two-step authentication. If the user has never enabled two-step authentication, the value is 0 (none).	

Example

This example requests the two-step authentication status of a user with object ID Office.1.1:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <GetUserMultiFactorAuthentication>
      <UserID>User.1.1</UserID>
    </GetUserMultiFactorAuthentication>
  </method>
</request>
```

The response returns a status of 0 (disabled). In addition, the `OTPNotifyMethod` value of 0 indicates that the user has never enabled two-step authentication:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <GetUserMultiFactorAuthentication>
      <UserMultiFactorAuthentication>
        <TwoWayAuthMode>0</TwoWayAuthMode>
        <OTPNotifyMethod>0</OTPNotifyMethod>
      </UserMultiFactorAuthentication>
    </GetUserMultiFactorAuthentication>
  </method>
  <_status>OK</_status>
  <_systime>2017-09-27T22:45:07</_systime>
  <!-- Cost 16 mill seconds. -->
</response>
```

InsertKeyRelations Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The InsertKeyRelations method adds a key relation to a SmartOffice contact record.

The InsertKeyRelations method is invoked using the method operation.

Note that the user inserting the key relation must have access to both contacts if office security is enabled.

Parameters

Parameter Name	Type	Required?	Description
ContactID	String	Yes	This is the object ID of the contact to which the key relation will be added.
RelatedToID	String	Yes	This is the object ID of the contact that will be the key relation.
RelationType	String	Yes	Valid values for this parameter are: 1 (Business), 2 (Family), 3 (Group Census), 4 (Personal) or 5 (Professional). Note that RelationType must correspond to the value of the Relation parameter.
Relation	String	Yes	The value of Relation must correspond to the value of RelationType.

			If RelationType is 1, valid values for Relation are: 0 (None), 1 (Associate), 9 (Employee), 10 (Employer), 16 (Fellow Employee), 28 (Partner), 40 (Subordinate), 41 (Superior). If RelationType is 2, valid values for Relation are: 0 (None), 2 (Aunt), 3 (Brother), 4 (Brother-in-Law), 5 (Cousin), 6 (Daughter), 7 (Daughter-in-Law), 8 (Domestic Partner), 11 (Ex-Husband), 12 (Ex-Spouse), 13 (Ex-Wife), 14 (Father), 15 (Father-in-Law), 17 (Fiancee), 19 (Granddaughter), 20 (Grandfather), 21 (Grandmother), 22 (Grandson), 23 (Husband), 24 (Mother), 25 (Mother-in-Law), 26 (Nephew), 27 (Niece), 31 (Sister), 32 (Sister-in-Law), 33 (Son), 34 (Son-in-Law), 35 (Spouse), 36 (Stepdaughter), 37 (Stepfather), 38 (Stepmother), 39 (Stepson), 42 (Uncle), 43 (Wife), 44 (Other), 55 (Child), 56 (Parent), 57 (Stepbrother), 58 (Stepsister), 59 (Grandchild), 60 (Fiance). If RelationType is 3, valid values for Relation are: 45 (Group Census). If RelationType is 4, valid values for Relation are: 0 (None), 18 (Friend), 29 (Referree), 30 (Referrer). If RelationType is 5, valid values for Relation are: 0 (None), 46 (Financial Advisor), 47 (Banker), 48 (Attorney), 49

			(Accountant), 50 (Broker), 51 (Financial Client), 52 (Dependent), 53 (Nominator), 54 (Nominee), 61 (Provider), 62 (Customer), 63 (Legal Client), 64 (Account Client), 65 (Broker Client), 66 (Trustee), 67 (Grantor), 68 (Assistant), 69 (Supervisor).

Example

The following example adds a family (husband) key relation to a contact:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <InsertKeyRelations>
      <ContactID>Contact.1.14</ContactID>
      <RelatedToID>Contact.1.38</RelatedToID>
      <RelationType>2</RelationType>
      <Relation>23</Relation>
    </InsertKeyRelations>
  </method>
</request>
```

The response returns the newly created key relations' object IDs:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <InsertKeyRelations>
      <Relation>
        <Contact>Relation.1.55</Contact>
        <RelatedTo>Relation.1.56</RelatedTo>
      </Relation>
    </InsertKeyRelations>
  </method>
  <_status>OK</_status>
  <_systime>2015-10-08T23:38:15</_systime>
  <!--Cost 43 mill seconds.-->
</response>
```

InsertRequirement Method

In this Topic Hide

- Overview
- Parameters
- Example

Overview

The `InsertRequirement` method generates requirements for a policy or pending case based on the underwriting guidelines defined for the associated carrier or product. This method uses carrier-level guidelines unless guidelines are defined at the product level, in which case the product-level guidelines take precedence.

The `InsertRequirement` method is invoked using the `method` operation.

Note: This method requires that a case manager be linked to the policy/pending case.

Parameters

Parameter Name	Type	Required?	Description
`holdingId`	Long	Yes	This is the holding ID of the policy or pending case for which requirements will be generated. To find this value in SmartOffice, use list layout customization to add the Holding ID column to the policy or pending case list.

Example

The following example request generates requirements for a pending case with a holding ID of 314:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <InsertRequirement>
      <holdingId>314</holdingId>
    </InsertRequirement>
  </method>
</request>
```

The response indicates that the requirements were generated successfully:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <InsertRequirement>
      <Result>
        <Status>Success</Status>
      </Result>
    </InsertRequirement>
  </method>
  <_status>OK</_status>
  <_systime>2023-06-01T11:18:20</_systime>
  <!-- Cost 764 mill seconds. -->
</response>
```

ProcessCustomChoice Method

In this Topic Hide

- Overview
- Parameters
- Examples
  - Inserting a Custom Choice
- Updating a Custom Choice
  - Deleting a Custom Choice

Overview

The `ProcessCustomChoice` method inserts, modifies or deletes custom lookup choices that appear in SmartOffice drop-down lists.

The `ProcessCustomChoice` method is invoked using the `method` operation.

Parameters

Parameter Name	Type	Required?	Description
`ChoiceID`	Integer	Yes	This is the ID that references the drop-down field and is found in the Lookup section of the API Dictionary.
`ChoiceIndex`	String	Yes (see description)	This is the index value of the lookup choice. This parameter is required only when you are modifying or deleting an existing lookup choice. To obtain the choice index value, you can insert a lookup choice using this method (the choice index is included in the XML response). Another way to obtain the choice index value is to use the `get` operation.
`Description`	String	Yes (see description)	This is the label for the lookup choice as it will appear in SmartOffice. Include this parameter when inserting or updating a lookup choice.
`Operation`	String	Yes	Valid values are: Insert, Update or Delete.

Examples

Inserting a Custom Choice

The following example inserts an "Internet" choice in the Source field of contact records. The Source lookup field is referred to as ContactSource in the API Dictionary; its ID is 178:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <ProcessCustomChoice>
      <ChoiceID>178</ChoiceID>
      <Description>Internet</Description>
      <Operation>Insert</Operation>
    </ProcessCustomChoice>
  </method>
</request>
```

The response returns the index value of the new choice in the `ChoiceIndex` element:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <ProcessCustomChoice>
      <Insert status="ok">
        <ChoiceIndex>65536</ChoiceIndex>
      </Insert>
    </ProcessCustomChoice>
  </method>
  <_status>OK</_status>
  <_systime>2016-12-13T20:01:35</_systime>
  <!-- Cost 88 mill seconds. -->
</response>
```

Updating a Custom Choice

The following example updates the "Internet" choice that we previously inserted, renaming it "Web":

```
<request version='1.0'>
  <header>...</header>
  <method>
    <ProcessCustomChoice>
      <ChoiceID>178</ChoiceID>
      <ChoiceIndex>65536</ChoiceIndex>
      <Description>Web</Description>
      <Operation>Update</Operation>
    </ProcessCustomChoice>
  </method>
</request>
```

The response returns the update status and the choice index value:

```
<response version="1.0">
  <header>
```

```
    <sessionClosed/>
  </header>
  <method>
    <ProcessCustomChoice>
      <Update status="ok">
        <ChoiceIndex>65536</ChoiceIndex>
      </Update>
    </ProcessCustomChoice>
  </method>
  <_status>OK</_status>
  <_systime>2016-12-13T20:09:54</_systime>
  <!-- Cost 48 mill seconds. -->
</response>
```

Deleting a Custom Choice

This example deletes the lookup choice inserted and updated in the preceding examples:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <ProcessCustomChoice>
      <ChoiceID>178</ChoiceID>
      <ChoiceIndex>65536</ChoiceIndex>
      <Operation>Delete</Operation>
    </ProcessCustomChoice>
  </method>
</request>
```

The response confirms the deletion status and the choice index value:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <ProcessCustomChoice>
      <Delete status="ok">
        <ChoiceIndex>65536</ChoiceIndex>
      </Delete>
    </ProcessCustomChoice>
  </method>
  <_status>OK</_status>
  <_systime>2016-12-13T20:14:52</_systime>
  <!-- Cost 39 mill seconds. -->
</response>
```

ProcessSmartPadArchive Method

In this Topic Hide

- Overview
- Parameters
- Examples
  - Archiving a SmartPad Entry
  - Restoring an Archived SmartPad Entry

Overview

The `ProcessSmartPadArchive` method archives a SmartPad entry or restores an archived SmartPad entry.

The `ProcessSmartPadArchive` method is invoked using the `method` operation.

Parameters

Parameter Name	Type	Required?	Description
`SmartPadID`	String	Yes	This is the object ID of the SmartPad entry to be archived or restored.
`NoteStatus`	String	Yes	Enter 0 (to restore an archived SmartPad entry) or 1 (to archive an entry).
`Category`	String	Yes (see description)	This is required when the `NoteStatus` parameter is set to 1. It is the SmartPad entry's classification for archiving purposes. Valid values: 1 (System-Generated Note), 2 (Old Note), 3 (Other), 4 (Incorrect Note).

`Reason`	String	Yes (see description)	This is required when the `Category` parameter is set to 3. It is a description of why the SmartPad entry is being archived.

Examples

Archiving a SmartPad Entry

The following request archives a SmartPad entry (identified by its object ID), assigns it to the Other archive category and provides a reason for the action:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <ProcessSmartPadArchive>
      <SmartPadID>SmartPad.1.87</SmartPadID>
      <NoteStatus>1</NoteStatus>
      <Category>3</Category>
      <Reason>No longer relevant</Reason>
    </ProcessSmartPadArchive>
  </method>
</request>
```

The response returns the object ID of the archived entry and the result of the operation:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <ProcessSmartPadArchive>
      <SmartPadArchive id="SmartPadArchive.1.87">SmartPad
Archived</SmartPadArchive>
    </ProcessSmartPadArchive>
  </method>
  <_status>OK</_status>
  <_systime>2015-10-02T16:31:16</_systime>
  <!--Cost 1017 mill seconds.-->
</response>
```

Restoring an Archived SmartPad Entry

The following example restores the SmartPad entry archived in the preceding example:

```
<request version='1.0'>
  <header>...</header>
  <method>
    <ProcessSmartPadArchive>
      <SmartPadID>SmartPadArchive.1.87</SmartPadID>
      <NoteStatus>0</NoteStatus>
    </ProcessSmartPadArchive>
  </method>
</request>
```

The response returns the object ID of the restored entry and the result of the operation:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <method>
    <ProcessSmartPadArchive>
      <SmartPad id="SmartPad.1.87">SmartPad Restored</SmartPad>
    </ProcessSmartPadArchive>
  </method>
  <_status>OK</_status>
  <_systime>2015-10-02T16:42:18</_systime>
  <!--Cost 222 mill seconds.-->
</response>
```

Sync Operation

In this Topic Hide

- Overview
- RMA Element
  - Phase Element
  - Query Element
  - Logic_Step Element
  - Return Element
- Objects Element
- Control Attributes
- Sync Example

Overview

The `sync` operation provides a way for clients application to synchronize data with SmartOffice. SmartIntegrator parses the request XML, and gets record-matching algorithm (RMA) logic, and then executes the object(s) one by one.

The `sync` element in the request XML contains two child elements:

- `rma`: This element defines the conditions by which SmartOffice should determine whether a match already exists in the database and what SmartOffice should do under various record-matching scenarios (e.g., match found, match not found, etc.).
- `objects`: This element defines the input of your `sync` operation, i.e., the data you want to sync to the SmartOffice database.

RMA Element

The `rma` element defines the record-matching algorithm SmartOffice will use in determining whether a matching object exists in the database. The following table describes the attributes of the `rma` element.

Attribute	Description
`start`	(Required) The starting phase of the RMA. The value is the ID of the phase.
`object`	(Required) The SmartOffice input object type that the RMA applies to (e.g., Contact).
[control attributes]	See Control Attributes.

Phase Element

An RMA is split into phases, each of which is represented by the `phase` element. The `rma` element must contain at least one `phase` element. Phases break the RMA into discrete sections containing their own queries and logic steps. The available attributes for the `phase` element are:

Attribute	Description
`id`	(Required) The phase identifier. This is a numerical value that is used to reference the phase in other portions of the RMA.
[control attributes]	See Control Attributes.

Query Element

Each `phase` element must contain a `query` element. A query defines the conditions (using the `condition` element) that will be applied when searching for a match in the SmartOffice database as well as the actions that will occur when the conditions are met or not met. The available attributes are:

Attribute	Description
[control attributes]	See Control Attributes.

Logic_Step Element

Within a `query` element, a logic step can be used to narrow down the results of a query. For example, if a query condition results in multiple matches (i.e., an ambiguity), a logic step can apply additional conditions on those results (using the `condition` element) to arrive at a match. The available attributes are:

Attribute	Description
`id`	(Required) The logic step identifier. This is a numerical value that is used to reference the logic step in other portions of the RMA.

Return Element

When creating an RMA, you can add control attributes that instruct SmartIntegrator to return a list of records that match the RMA.
The `return` element is where you define the properties that you want returned.

Objects Element

The `objects` element can be used to define the data you want to sync to the SmartOffice database. One or more objects can be defined here.

Control Attributes

Control attributes can be used with
the `rma`, `phase`, `query` and `logic_step` elements. Control attributes define

the actions that will occur when the conditions in the RMA are applied. Attributes specified for child elements override attributes specified in parent elements. It is recommended to use control attributes with the `query` and `logic_step` elements until you become more familiar with how the `sync` operation works.

The available control attributes are summarized in the following table:

Attribute	Description
`match`	Defines what happens when a single match is found.
`ambiguity`	Defines what happens when more than one match is found.
`no-match`	Defines what happens when no match is found.
`insufficent-data`	Defines what happens when the input defined in the `objects` element does not contain the properties required by the RMA.

The available values for the control attributes described above are:

Attribute Value	Description
`ignore`	Ignore processing/do nothing.
`insert`	Insert the objects defined in the `objects` element into the SmartOffice database as new records.
`update`	Update the matching record(s) in the SmartOffice database with the data specified in the `objects` element.
`delete`	Delete the matching record(s) in the SmartOffice database.

`return`	Return all matching objects with the properties defined in the `return` element.
`exit`	Exit from the current step.
[phase ID] or [step ID]	Execute the specified phase/step.

Sync Example

In this example, the record-matching algorithm defined in the `rma` element contains two phases. The first phase contains a query defining two conditions for a match (last name and tax ID) and a logic step defining an additional condition (first name) if the query returns an ambiguous result.

```
<request version="1.0">
  <header>...</header>
  <sync>
    <rma object="Contact" start="phase1">
      <phase id="phase1">
        <query no-match="phase2" match="update" ambiguity="step1">
          <condition>
            <expr prop="LastName" op="eq">
              <v type="ref">Contact.LastName</v>
            </expr>
          </condition>
          <condition>
            <expr prop="TaxID" op="eq">
              <v type="ref">Contact.TaxID</v>
            </expr>
          </condition>
        </query>
        <logic_step id="step1" no-match="phase2" match="update"
        ambiguity="return">
          <condition>
            <expr prop="FirstName" op="eq">
              <v type="ref">Contact.FirstName</v>
            </expr>
          </condition>
        </logic_step>
      </phase>
      <phase id="phase2">
        <query no-match="insert" match="update" ambiguity="return">
          <condition>
            <expr prop="TaxID" op="eq">
              <v type="ref">Contact.TaxID</v>
```

```
            </expr>
          </condition>
        </query>
      </phase>
      <return>
        <Contact>
          <LastName/>
          <FirstName/>
        </Contact>
      </return>
    </rma>
    <objects>
      <Contact>
        <LastName>Ackerman</LastName>
        <FirstName>Joseph</FirstName>
        <TaxID>111111111</TaxID>
        <WebAddresses>
          <WebAddress>
            <WebAddressType>1</WebAddressType>
            <Address>jackerman@example.com</Address>
          </WebAddress>
        </WebAddresses>
      </Contact>
    </objects>
  </sync>
</request>
```

Breaking down this example, let's look at the query first:

```
<query no-match="phase2" match="update" ambiguity="step1">
  <condition>
    <expr prop="LastName" op="eq">
      <v type="ref">Contact.LastName</v>
    </expr>
  </condition>
  <condition>
    <expr prop="TaxID" op="eq">
      <v type="ref">Contact.TaxID</v>
    </expr>
  </condition>
</query>
```

The attributes of the `query` element specify what will happen when the conditions are applied. If there is no match, phase 2 of the RMA will begin. If there is a single match, the contact will be updated with the data specified in the `objects` element. If there is ambiguity (i.e., more than one match), the RMA will go to logic step 1 to further narrow down the matches.

Let's look at that logic step:

```
<logic_step id="step1" no-match="phase2" match="update" ambiguity="return">
  <condition>
```

```
    <expr prop="FirstName" op="eq">
      <v type="ref">Contact.FirstName</v>
    </expr>
  </condition>
</logic_step>
```

The logic step refines the query by taking the query results and applying an additional condition (in this case, checking for a match on the first name). If no match is found, phase 2 of the RMA will begin. If a match is found, the contact will be updated with the data specified in the `objects` element. If there is still ambiguity even after matching on the first name, the request will just return a list of the matching records.

In the second phase of the RMA, the query contains only one condition, a match on the tax ID:

```
<query no-match="insert" match="update" ambiguity="return">
  <condition>
    <expr prop="TaxID" op="eq">
      <v type="ref">Contact.TaxID</v>
    </expr>
  </condition>
</query>
```

Again, the attributes of the `query` element specify what will happen after the conditions are applied. If there is no match, the contact data specified in the `objects` element will be inserted into the database as a new contact. If there is a single match, the contact will be updated with the data specified in the `objects` element. If there is ambiguity (i.e., more than one match), the request will return a list of the matching records and do nothing else.

The `return` element defines what data SmartIntegrator will return about each object when instructed to return data. In our example, when the XML specifies that results should be returned, SmartIntegrator will return the last name and first name of all contacts that were a match.

```
<return>
  <Contact>
    <LastName/>
    <FirstName/>
  </Contact>
</return>
```

Finally, the `objects` element in our example contains the input data for the sync operation. Here, we define the object type (`Contact`) and the properties of the object that we want to process using the RMA.

```
<objects>
  <Contact>
    <LastName>Ackerman</LastName>
    <FirstName>Joseph</FirstName>
    <TaxID>111111111</TaxID>
    <WebAddresses>
      <WebAddress>
        <WebAddressType>1</WebAddressType>
        <Address>jackerman@example.com</Address>
      </WebAddress>
    </WebAddresses>
  </Contact>
</objects>
```

Referring Objects/Properties

SmartIntegrator provides an option to refer objects/properties in the same request. The user can refer to the object based on its reference ID. A property of the object can be referred to as `@ReferenceID.PropertyName`.

Using `@ReferenceID` without a property name retrieves the ID of the object.

The following example uses the `insert` operation to add a contact object to the database as well as a SmartPad note object.
The `refid="john"` attribute of the `Contact` element assigns a reference to the object. That reference is then used to create a SmartPad object linked to that contact.

```
<request version='1.0'>
  <header>...</header>
  <insert>
    <Contact refid="john">
      <LastName>Doe</LastName>
      <FirstName>John</FirstName>
      <Title>Dr</Title>
      <TaxID>111111111</TaxID>
      <Person>
        <Dob>1950-02-27</Dob>
      </Person>
    </Contact>
```

```
    <SmartPad>
      <Keywords type="ref">@john.TaxID</Keywords>
      <Note>How are you?</Note>
      <ContactID type="ref">@john</ContactID>
    </SmartPad>
  </insert>
</request>
```

Conditions

In this Topic Hide

- Overview
- XML Structure
- Expression
- Value
- Operators
- Functions
- Child Object Properties
- Sort

Overview

Conditions are defined in the XML format, and the structure of the XML is similar to SQL queries. Conditions are used in `search` and `sync` operations.

XML Structure

Here is an an example of a condition that specifies the object property to be evaluated (Contact.Name), the operation to be used for evaluating the property (like), and the value for the condition (A%):

```
<condition>
  <expr prop="Contact.Name" op="like">
    <v>A%</v>
  </expr>
</condition>
```

Note the following:

- A condition element contains a condition definition. Conditions can be nested, i.e., one condition can contain another condition.
- The expr element defines a minimal Boolean expression.
- The and and or elements (not shown in the example) can be used with expressions and conditions to represent Boolean operators.

Expression

The expr element defines a minimal Boolean expression. The left side of the expression is a property, while the right side can have one or more constant values or a property. The left-side information is defined by the following expr element attributes:

Attribute	Description	Examples
prop	Left-side property name in the format object.property.sub-property.	Contact.Type Contact.PreferredPhone.AreaCode
op	The operator that you want to use for the expression. See Operators.	
fn	An optional function that can be applied to the specified property to further refine the search. See Functions.	

Value

Once you define the expr element and its attributes, use the v element to define the value or values for the right side of the expression.

Note: When an expression uses the isnull or not-null operator, do not include a v element.

The v element can have the following attributes:

Attribute	Description	Notes
type	The value type. Acceptable types are:	When no type attribute is set, SmartIntegrator assumes that the type is const.

	• prop : property name • const : constant value • ref : reference to current object's property value	The const type must be used when the expression uses any of these operators: between, in, like, starts, ends, contain. The ref type is for referring objects/properties.
fn	See Functions.	

Operators

The following table summarizes the operators that can be used as attributes for the expr element.

Operator	Description	Examples
eq	Equal to	
gt	Greater than	
ge	Greater than or equal to	

lt	Less than	
le	Less than or equal to	
ne	Not equal to	

between	Between two values (must be integers)	Find contacts whose Type is a value between 1 and 10:
		`<expr prop="Contact.Type" op="between">`
		`<v>1</v>`
		`<v>10</v>`
		`</expr>`

like	Matching a specified pattern. Accepts the wildcards % (for no, one or multiple characters) and _ (for only one character) in the v element. Note that your application is responsible for escaping special characters in the string value.	Find contacts whose name starts with "A" followed by no, one or multiple characters:
		`<expr prop="Contact.Name" op="like">`
		`<v>A%</v>`
		`</expr>`

contains	Contains a text string	Find contacts whose name contains the characters "abc" (in SQL syntax, this would be name like '%abc%'):
		`<expr prop="Contact.Name" op="contains">`
		`<v>abc</v>`
		`</expr>`

starts	Starts with a text string	Find contacts whose name begins with the letter "A" (in SQL syntax, this would be name like 'A%'): <expr prop="Contact.Name" op="starts"> <v>A</v> </expr>
ends	Ends with a text string	Find contacts whose name ends with the letter "A" (in SQL syntax, this would be name like '%A'): <expr prop="Contact.Name" op="ends"> <v>A</v> </expr>
in	Test if a property is from a list of values. Only simple type values are allowed.	Find contacts who Type is 1, 2 or 3 (in SQL syntax, this would be type in (1,2,3)'): <expr prop="Contact.Type" op="in"><v>1</v> <v>2</v> <v>3</v> </expr>

isnull	Is null. Note that this operator takes no v element.	Find contacts with no first name (in SQL syntax, this would be name is null): <expr prop="Contact.FirstName" op="isnull" />
not-between	Not between	
not-in	Not in	
not-null	Is not null. Note that this operator takes no v element.	
not-like	Not like	
not-contains	Not contains	
not-starts	Not starts	
not-ends	Not ends	
any	Any of the child object matches with the given data (supported only by the sync operation's logic_step condition.)	

Functions

To refine a query, you can apply a function to a property by adding the fn attribute to the expr element. The following example uses the MONTH function to find contacts with a date of birth in March:

```
<expr prop="Contact.Dob" op="eq" fn="MONTH">
  <v>3</v>
```

```
</expr>
```

SmartIntegrator supports only those SQL functions that are supported by the SmartOffice framework. The queried object must support the function in order for SmartIntegrator to return a result. Functions can only be applied to properties.

The available functions are:

- YEAR
- MONTH
- DAY (Day of montth, i.e., 1–31)
- HOUR
- MINUTE
- SECOND
- QUARTER (Quarter of year)
- WEEKOFYEAR (Week of year)
- DAYOFYEAR
- DAYOFWEEK
- UCASE (Upper case)
- LCASE (Lower case)

Child Object Properties

Searchable properties of child objects can also be specified in an expression. Relationships must be defined by the sub-type and foreign keys.

This sample expression looks for all contacts with a postal code of 91106.

```
<expr prop="Contact.PreferredAddress.Postal" op="eq">
  <v>91106</v>
</expr>
```

Sort

The sort element defines the object property by which data returned by SmartIntegrator will be sorted. Only searchable properties can be sorted.

Within the sort element, use the item element with the following attributes to specify the sorting:

Attribute	Description
prop	The name of a property. This is a short name and does not include an object name.
order	Accepts one of two values: asc (ascending) or desc (descending). The default is asc.

Example:

```
<sort>
  <item prop="AreaCode" order="asc"/>
</sort>
```

Sorting is supported up to two levels of properties. For example:

```
<sort>
  <item prop="Assignment.Contact" order="asc"/>
</sort>
```

Error Handling

In this Topic Hide

- Overview
- Status Element
- Error Element
- Error Behavior
- Error Code List

Overview

This topic describes errors you may encounter when sending requests to SmartIntegrator.

Status Element

The _status element at the end of an XML response indicates whether SmartIntegrator encountered an error processing a request.

- A status of OK means no error occurred.
- A status of error means at least one error occurred. Information about the error is included in the error element in the response.

Error Element

The error element in a response XML contains the error code and description. Here is an example of an invalid search operation request:

```
<request version="1.0">
  <header>
    <office/>
    <user/>
    <password/>
    <keepsession>true</keepsession>
  </header>
  <search total="true" pagesize="3">
    <object>
      <Contact>
        <LastName/>
```

```
        <FirstName/>
      </Contact1>
    </object>
    <condition>
      <expr prop='LastName' op='starts'>
        <v>A</v>
      </expr>
    </condition>
  </search>
</request>
```

The typo in the closing tag for the Contact element generates the following error response:

```
<response version="1.0">
  <header>
    <sessionClosed/>
  </header>
  <error code="1001">Invalid XML format. The end-tag for element type
"Contact" must end with a '>' delimiter.</error>
</response>
```

Clients must parse the entire response to find out if an error occurred.

Error Behavior

There are two types of errors that SmartIntegrator returns:

- **System errors** are internal errors caused by existing issues or the environment. When this happens, the stack trace is printed out.
- **Process errors** are caused by ill-formed request XML.

Error codes 2001, 2002 and 2003 denote login and session errors. The process is aborted if one of those errors occurs.

Error 1001 is an XML syntax error that also stops processing.

All other errors produce an error element generated inside the element that caused the error, but SmartIntegrator continues to process the rest of the request. All unfinished operations in a transaction element are canceled if an error happens inside the transaction.

The following example shows the different error behavior inside and outside of a transaction. Here is the request:

```
<request version="1.0">
  <header>...</header>
  <transaction>
    <get>
```

```
      <Contact id="Contact.">
        <FullName/>
      </Contact>
    </get>
    <get>
      <Contact id="Contact.65854.3">
        <FullName/>
      </Contact>
    </get>
  </transaction>
  <get>
    <Contact id="Contact.">
      <FullName/>
    </Contact>
  </get>
  <get>
    <Contact id="Contact.65854.3">
      <FullName/>
    </Contact>
  </get>
</request>
```

The response shows that the error inside the transaction element caused all of the get requests in that transaction to error, while the valid get request that was not part of the transaction was processed:

```
<response version="1.0">
  <header>...</header>
  <transaction>
    <get>
      <Contact id="Contact." _status="error">
        <error code="3004">Invalid Object Id.</error>
      </Contact>
    </get>
  </transaction>
  <get>
    <Contact id="Contact." _status="error">
      <error code="3004">Invalid Object Id.</error>
    </Contact>
  </get>
  <get>
    <Contact id="Contact.65854.3">
      <FullName>Ackerman, Joseph</FullName>
    </Contact>
  </get>
  <_status>error</_status>
</response>
```

Error Code List

Code	Symbol	Message
System Errors		
1	ERROR_SYSTEM	Internal error.
2	ERROR_ILLEGEL_ARGUMENT	Illegal Argument Exception. %1
3	ERROR_INVALID_DATA_TYPE	Invalid data type %1.
4	ERROR_UNSUPPORTED_TYPE	Unsupported type %1.
5	ERROR_FAILED_TO_CREATE_OBJECTID	Failed to create object ID (%1).
6	ERROR_MISMATCH_OBJECTID	Object ID (%1) does not match with object (%2).
XML Errors		
1001	ERROR_XML_INVALID_FORMAT	Invalid XML format.
1002	ERROR_XML_INVALID_TAG_NAME	Invalid Tag name (<%1>).
Login/Session Errors		
2001	ERROR_INCOMPATIBLE_VERSION	Request version %2 is not match with Catalog Version %1
2002	ERROR_INVALID_LOGIN_INFO	Login failed.
2003	ERROR_INVALID_SESSION	Invalid Session ID.

2004	ERROR_NO_RIGHT_TO_SWITCH_USER	Login user does not have sufficient rights to impersonate to other users.
2005	ERROR_SWITCH_USER_NOT_FOUND	Office "%1" or user "%2" cannot be found.
2006	ERROR_INVALID_OFFICE_INFO	Invalid office info.
2007	ERROR_INVALID_USER_INFO	Invalid user info.
2008	ERROR_UNIQUEID_LOGIN_DISABLED	The UnqiueID Login is disabled, Please check the configuration file.
2009	ERROR_NO_RIGHT_TO_SWITCH_OFFICE	Login user does not have enough right to impersonate to other offices.
Object Errors		
3001	ERROR_OBJECT_NOT_EXIST	Object is not found (<%1>).
3002	ERROR_OBJECT_NOT_SEARCHABLE	Object is not Searchable (<%1>).
3003	ERROR_OBJECT_READONLY	Object is read-only (<%1>).
3004	ERROR_OBJECT_ID	Invalid Object Id.
3005	ERROR_OBJECT_NO_ID	Object Id value is null or blank.

Property Errors		
4001	ERROR_PROPERTY_NOT_EXIST	Property is not found (<%1>).
4002	ERROR_PROPERTY_NOT_SEARCHABLE	Property is not Searchable (<%1>).
4003	ERROR_PROPERTY_READONLY	Property is read only (<%1>).
4004	ERROR_PROPERTY_INVALID_OPTION	Invalid property option (%1).
Keyword Errors		
5001	ERROR_INVALID_FUNCTION	Invalid function (%1).
5002	ERROR_INVALID_OPERATOR	Invalid operator (%1).
5003	ERROR_INVALID_BOOLEAN	Invalid Boolean value (%1). "true" or "false".
Condition Errors		
6001	ERROR_COND_PROPERTY_NOT_ALLOWED	Property is not allowed in search condition (%1).
6002	ERROR_COND_DATA_TYPE_NOT_ALLOWED	Property is not allowed in search condition (%1) due to its data type (%2).
6003	ERROR_COND_NOT_EMBEDDED	Property is not embedded object ("%1" in "%2").

6004	ERROR_SEARCH_NO_OBJECT	No object is specified to search.
6005	ERROR_EXPR_NO_PROPERTY	No property is specified in expression.
6006	ERROR_SEARCH_INVALID_SEARCHID	Invalid search ID (%1).
6007	ERROR_COND_OBJECT_NOT_IN_JOIN	Embedded object is not include in join ("%1"). Maximum depth level of embedded object for search condition is 4. The depth of given property name may exceed the limit.
6008	ERROR_OPERATOR_NOT_ALLOWED	Operator (%1) is not allowed for %2 (%3).
6009	ERROR_INVALID_LINKID	Invalid Link ID (%1). Target table primary keys cannot be referred by main table's Link ID.
6010	ERROR_INSUFFICENT_SEARCH_DATA	Insufficient search data.
Method Errors		
7001	ERROR_METHOD_NOT_EXIST	Method is not found (%1).
7002	ERROR_METHOD_INVOKE	Invoke method %1 failed.
Sync Errors		
8001	ERROR_RMA_NOT_EXIST	RMA is not found (%1).

8002	ERROR_IMPROPER_RETURN_OBJECT	Improper return object.
Data Access Errors		
10001	ERROR_DATA_ACCESS_DENIED	The user does not have permission to access this module. Please contact an office administrator for further assistance.

API Dictionary

In this Topic Hide

- Overview
- Accessing the API Dictionary

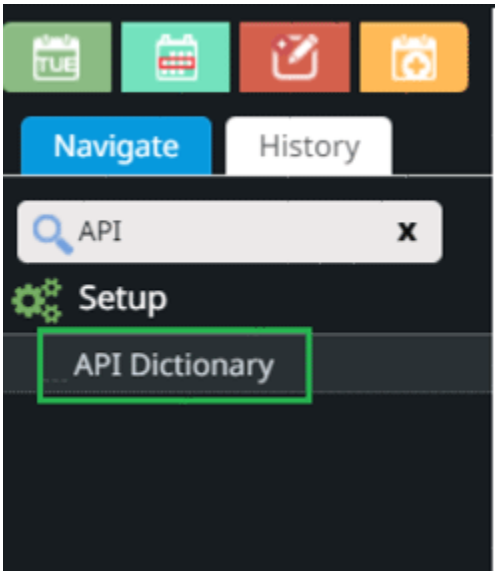
Overview

The API Dictionary contains comprehensive descriptions of all SmartOffice objects, properties and lookups that can be accessed through SmartIntegrator. Developers can refer to the API Dictionary to evaluate available data elements and understand the SmartOffice data model.

Accessing the API Dictionary

The API Dictionary is a single HTML page that is generated by SmartIntegrator in real time when you access it.

All SmartOffice users with administrator rights can access the API Dictionary by selecting **Setup** > **API Dictionary** from the SmartOffice side menu (to find it quickly, type "API" in the side menu search box).



Note: If you do not see the API Dictionary option under Setup, contact the SmartOffice team and request that it be enabled for your office.

Request/Response Collection and Using Postman

In this Topic Hide

- Overview
- Postman Setup
- Postman Collection

Overview

The Postman app is recommended for API development and testing. The SmartOffice team provides developers with a Postman collection to use as part of the developer onboarding process.

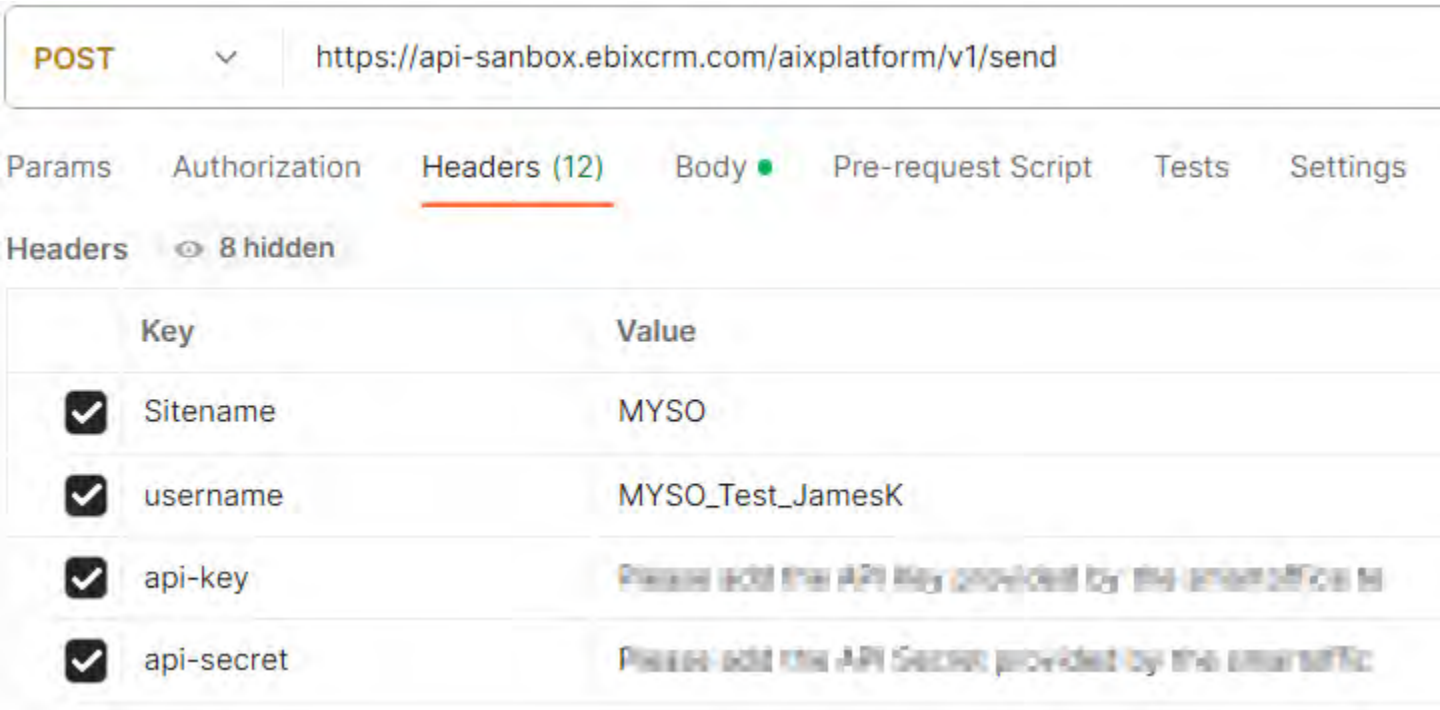
This topic describes how to set up Postman to send requests to SmartIntegrator.

Download Postman from the Postman website. Documentation for Postman app is available at the Postman Learning Center.

Postman Setup

The headers required to execute request using Postman are:

- sitename
- username
- api-key
- api-secret



Select the **POST** method in Postman to send requests to URL provided in your Postman collection. Enter your xml request in **raw** format under Body.

POST https://api-sandbox.ebixcrm.com/aixplatform/v1/send

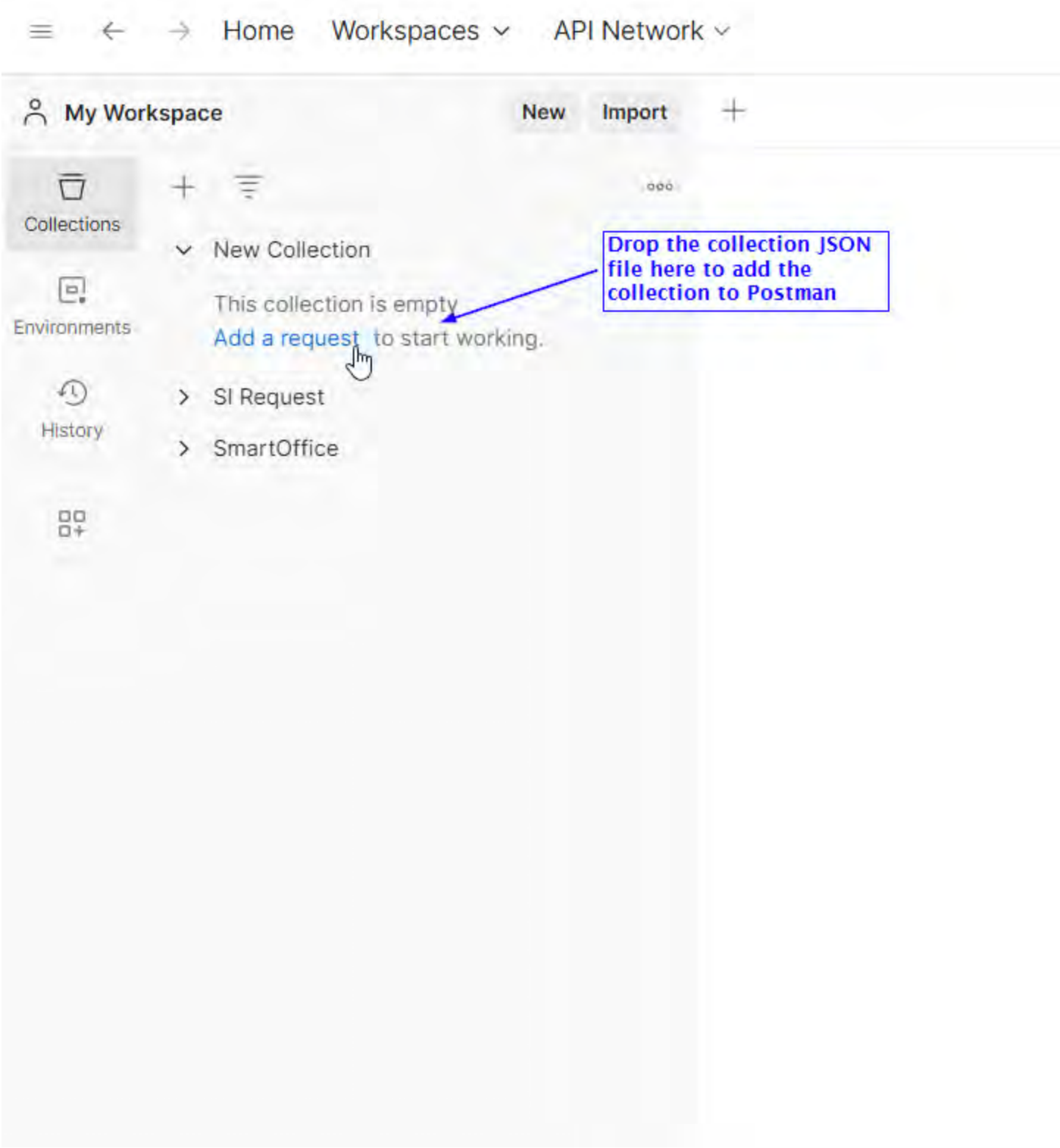
Params Authorization Headers (11) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL XML

```xml
<request version="1.0">
    <header>
        <office></office>
        <user></user>
        <password></password>
    </header>
    <search>
        <object>
            <Contact>
                <LastName/>
                <FirstName/>
            </Contact>
        </object>
        <condition>
            <expr prop='LastName' op='starts'>
                <v>A</v>
            </expr>
        </condition>
    </search>
```

Postman Collection

The collection of sample Postman requests provided to you during the onboarding process can be imported into Postman and updated to use as required.


Home
Workspaces
API Network
My Workspace
New
Import
Collections
Environments
History
New Collection
This collection is empty
Add a request to start working.
SI Request
SmartOffice
Drop the collection JSON file here to add the collection to Postman

Key Rotation

The API key and API secret provided to you as part of the developer onboarding process will be rotated every 90 days.

A new key will be generated automatically seven (7) days before the existing key expires. You can use your current key or the new key to execute the requests via Postman until the existing key expires.

Once the existing key expires, it will no longer work for sending requests to SmartIntegrator. You will need to pass the new key as the value of **api-key** and **api-secret** respectively in Postman's Header section to continue executing requests.

Frequently Asked Questions

In XML requests, the `header` element contains `office`, `user` and `password` elements. What data goes in these elements?

Although they must be included in your XML requests, the `office`, `user` and `password` elements should be empty, as described in the Request/Response section.

What happens if the connection is broken while executing multiple insert or update requests? Will it generate bad data?

The best practice is to enclose all `insert` and `update` requests inside the `transaction` element. This will roll back all operations if the connection is broken before completion of all operations. If the `transaction` element is not used, operations completed while the connection was active will be committed.

Are then instances in which a particular SmartOffice field is available only in the client's instance and not in the sandbox environment?

No.

Why do I get the error "Unable to parse input string" when I try to query by HoldingType?

HoldingType is a Lookup-type column. You need to pass a valid LookUp value in the condition to get a result.

When I submit a `search` request, why doesn't the response not contain the search ID?

You must add the `<keepsession>true</keepsession>` tag in the `header` section of the request to receive a search ID in the response.